

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

федеральное государственное автономное образовательное
учреждение

высшего образования

«Санкт-Петербургский политехнический университет Петра
Великого»

(ФГАОУ ВО «СПБПУ»)

Проект

Допущен к защите

Руководитель

Дирекции образовательных
программ

_____ И.М. Зайченко

« _____ » _____ 2026 г.

ДИПЛОМНЫЙ ПРОЕКТ(РАБОТА)

Тема Разработка информационной системы для управления проектами и
задачами

по специальности 09.02.07 Информационные системы и программирование
код и наименование

Студент(ка) гр. 2290907/1094

_____ (подпись)

_____ (ФИО)

Руководитель

_____ (подпись)

_____ (ФИО)

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ОБЩАЯ ЧАСТЬ	4
1.1 Анализ предметной области	4
1.2 Постановка задачи.....	6
1.3 Анализ рынка существующих решений	8
1.4 Обоснование и выбор инструментальных средств разработки.....	13
2 СПЕЦИАЛЬНАЯ ЧАСТЬ.....	21
2.1 Проектирование архитектуры системы	21
2.2 Проектирование базы данных.....	30
2.3 Разработка серверной части	34
2.4 Разработка интерфейса	36
2.5 Тестирование и отладка программного продукта	40
3 ЭКОНОМИЧЕСКАЯ ЧАСТЬ.....	44
3.1 Область применения программного продукта и его преимущества перед аналогичным программным продуктом	44
3.2 Трудоемкость разработки программного продукта, квалификация исполнителя и его оклад.....	44
3.3 Расчет затрат на разработку	46
3.4 Расчет цены и прибыли	51
ЗАКЛЮЧЕНИЕ	54
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	56
Приложение А (справочное) Код моделей	58
Приложение Б (справочное) Код Entity Framework Core конфигураций	67
Приложение В (справочное) Код среза функции регистрации	72
Приложение Г (справочное) Код реализации аутентификации и авторизции	74
Приложение Д (справочное) Код тестов.....	76

ВВЕДЕНИЕ

В современных условиях цифровизации всё больше организаций переходят к проектной модели управления, при которой эффективность работы напрямую зависит от качества планирования, распределения задач и контроля выполнения процессов. Проектные команды ежедневно работают с большим объёмом информации: данными о проектах, задачах, сроках выполнения, исполнителях и статусах работ. Использование разрозненных инструментов, таких как электронные таблицы, почта и текстовые документы, затрудняет организацию совместной работы, приводит к ошибкам, потере времени и снижению производительности сотрудников.

Для повышения эффективности управления проектами востребованными становятся автоматизированные информационные системы, обеспечивающие централизованное хранение данных, автоматизацию бизнес-процессов и разграничение прав доступа пользователей. Особую актуальность приобретает разработка серверной части таких систем в виде API-интерфейсов, позволяющих обеспечить гибкое взаимодействие с различными клиентскими приложениями и возможность дальнейшего масштабирования программного продукта.

В рамках работы основное внимание уделяется проектированию и реализации серверной части приложения, обеспечивающей обработку запросов, хранение и управление данными, выполнение бизнес-логики и поддержку ролевой модели доступа.

Целью дипломного проекта является разработка информационной системы для управления проектами и задачами, обеспечивающей централизованное хранение информации, автоматизацию процессов управления задачами и взаимодействие с клиентскими приложениями посредством API.

1 ОБЩАЯ ЧАСТЬ

1.1 Анализ предметной области

1.1.1 Общая характеристика

Управление проектами представляет собой совокупность процессов планирования, распределения и контроля выполнения задач, направленных на достижение целей организации в рамках различных ограничений.

В современных организациях проектный подход широко используется в разработке программного обеспечения, бизнес-аналитике и других сферах, где требуется координация работы нескольких участников. Основой данного подхода является декомпозиция проекта на отдельные задачи с последующим распределением их между исполнителями и контролем выполнения.

Эффективное управление проектами требует явного учета исполнителей, задач и их статусов, что делает разработку специализированных информационных систем актуальной.

1.1.2 Основные сущности

В рамках предметной области можно выделить следующие ключевые сущности:

Проект — основная структурная единица, объединяющая задачи и участников. Проект имеет название, описание, приоритет и статус выполнения. Он задаёт контекст выполнения всех связанных задач и определяет группу участников под заданные задачи.

Задача — элемент работы внутри проекта, описывающий конкретную единицу выполнения. Задача имеет название, описание, срок выполнения, статус и может быть назначена конкретному исполнителю.

Исполнитель проекта — пользователь, включённый в состав участников конкретного проекта и доступный для назначения на задачи в рамках этого проекта.

Менеджер проекта — пользователь, выполняющий организационную работу над участниками проекта.

1.1.3 Основные процессы

В рамках предметной области выделяются основные процессы жизненного цикла проекта:

- создание проекта — формирование проекта с указанием его основных характеристик;
- назначение участников проекта — формирование пула исполнителей, доступных для выполнения задач;
- создание задач — формирование задач внутри проекта и определение их параметров;
- распределение задач — назначение исполнителей из пула проекта на конкретные задачи;
- выполнение задач — выполнение работ исполнителями и обновление статуса задач;
- контроль выполнения — отслеживание состояния задач и заперт завершения проекта, при наличии в нём активных задач;
- завершение проекта — фиксация состояния проекта и задач при условии, что все задачи завершены.

Данные процессы формируют непрерывный цикл управления проектной деятельностью.

1.1.4 Основные информационные данные

В процессе управления проектами формируется и обрабатывается значительный объём данных, включающий:

- данные о пользователях и их ролях;
- информация о проектах;
- данные о задачах;

- связи между проектами, задачами и участниками;
- изменения статусов и назначений.

Основные информационные данные включают создание и обновление проектов и задач, назначение исполнителей и изменение статусов выполнения задач.

Обработка данных должна обеспечивать их целостность, согласованность и актуальность в рамках системы.

1.1.5 Вывод по анализу предметной области

Анализ предметной области показал, что управление проектами представляет собой совокупность взаимосвязанных процессов, направленных на организацию и контроль выполнения задач в рамках проектов. Основу предметной области составляют проекты, задачи и участники, взаимодействие которых определяется ролями пользователей и установленными ограничениями.

В процессе управления проектами выполняются операции создания проектов, формирования состава исполнителей, распределения задач и контроля их выполнения. При этом важное значение имеет хранение и обработка данных о проектах, задачах, пользователях и их взаимосвязях.

Таким образом, предметная область характеризуется необходимостью централизованного хранения информации, контроля состояний задач и проектов, а также управления взаимодействием участников. Это обуславливает необходимость разработки информационной системы, предназначенной для автоматизации процессов управления проектной деятельностью.

1.2 Постановка задачи

В результате анализа предметной области была выявлена необходимость разработки информационной системы управления проектами,

предназначенной для автоматизации процессов создания задач, группировки их по проектам и распределения между исполнителями.

Разрабатываемая система должна обеспечивать централизованное хранение информации о проектах, задачах и пользователях, а также предоставлять средства управления взаимодействием участников проектной деятельности. Основной целью системы является повышение эффективности управления проектами за счёт автоматизации основных операций, связанных с планированием, назначением и контролем выполнения задач.

Система должна поддерживать разграничение прав доступа в зависимости от роли пользователя. Менеджер проекта должен обеспечивать организацию работы внутри проекта: формировать состав исполнителей, создавать задачи, назначать исполнителей и отслеживать состояние выполнения задач. Исполнитель должен иметь доступ к назначенным ему задачам и возможность изменять их статус в процессе выполнения.

Разрабатываемая система должна обеспечивать работу с проектами и задачами с учётом установленных бизнес-ограничений. Исполнители задач могут назначаться только из состава участников проекта. Завершение проекта должно быть возможно только при отсутствии активных задач. Для завершённых проектов и задач должны действовать ограничения на изменение данных. Также система должна обеспечивать корректную обработку связанных данных при удалении пользователей, проектов и задач.

Дополнительно система должна обеспечивать хранение информации о статусах задач и проектов, приоритетах проектов, назначениях исполнителей и изменениях состояния объектов. Все данные должны обрабатываться с соблюдением требований целостности и актуальности.

Результатом выполнения работы должна стать информационная система, предоставляющая API для взаимодействия с данными предметной области и обеспечивающая автоматизацию процессов управления проектной деятельностью.

1.3 Анализ рынка существующих решений

1.3.1 Общая характеристика существующих систем

На современном рынке представлено большое количество информационных систем, предназначенных для управления проектами и задачами. Подобные решения используются для планирования работ, распределения задач между участниками команды, контроля сроков выполнения и организации взаимодействия пользователей.

Большинство существующих систем предоставляют базовые возможности управления проектами, однако отличаются подходами к организации рабочих процессов, уровнем сложности, гибкостью настройки и ориентацией на определённые категории пользователей. Некоторые решения ориентированы на крупные корпоративные проекты, другие — на небольшие команды.

Для анализа были рассмотрены наиболее распространённые системы управления проектами: Jira, Trello, YouTrack и Bitrix24.

1.3.2 Jira

Jira является одной из наиболее распространённых систем управления проектами и задачами, разработанной компанией Atlassian. Данное решение широко применяется в сфере разработки программного обеспечения и поддерживает различные методологии управления проектами, включая Scrum и Kanban.

Система предоставляет возможности создания проектов, ведения задач, распределения исполнителей, настройки рабочих процессов и формирования отчетности. Также Jira поддерживает гибкую настройку ролей пользователей и интеграцию с большим количеством внешних сервисов.

К преимуществам Jira относятся:

- высокая гибкость настройки;
- развитая система управления задачами;

- поддержка Agile-методологий;
- наличие инструментов аналитики и отчетности.

Недостатками системы являются:

- высокая сложность настройки и сопровождения;
- перегруженность интерфейса;
- избыточность функциональности для небольших проектов;
- часть возможностей доступна только в коммерческих версиях.

1.3.3 Trello

Trello представляет собой систему управления задачами, основанную на использовании канбан-досок. Основной принцип работы заключается в визуальном распределении задач по колонкам, отражающим этапы выполнения.

Пользователи могут создавать карточки задач, назначать исполнителей, устанавливать сроки выполнения и отслеживать текущее состояние задач. Система отличается простотой использования и минимальными требованиями к обучению пользователей.

К преимуществам Trello относятся:

- простой и интуитивно понятный интерфейс;
- удобное визуальное представление задач;
- быстрое внедрение в рабочие процессы;
- поддержка совместной работы.

Недостатками системы являются:

- ограниченные возможности управления сложными проектами;
- слабая поддержка строгих бизнес-правил;
- недостаточная детализация ролей и прав доступа;
- ограниченная функциональность в бесплатной версии.

1.3.4 YouTrack

YouTrack — система управления проектами и отслеживания задач, разработанная компанией JetBrains. Данное решение ориентировано преимущественно на команды разработки программного обеспечения.

Система предоставляет возможности управления проектами, ведения задач, настройки рабочих процессов, контроля сроков и формирования отчетности. Особенностью YouTrack является развитая система поиска и поддержка автоматизации процессов.

К преимуществам YouTrack относятся:

- гибкая настройка рабочих процессов;
- удобная система поиска и фильтрации задач;
- поддержка автоматизации;
- интеграция со средствами разработки.

Недостатками системы являются:

- высокая сложность для неподготовленных пользователей;
- сложность первоначальной настройки;
- ориентация преимущественно на IT-проекты;
- наличие функциональности, не востребованной в небольших системах управления проектами.

1.3.5 Bitrix24

Bitrix24 представляет собой комплексную корпоративную платформу, включающую средства управления проектами, задачами, коммуникациями и клиентскими данными.

Система поддерживает создание проектов, постановку задач, назначение исполнителей, совместную работу и обмен сообщениями между участниками. Также платформа содержит встроенные средства документооборота и CRM-функциональность.

К преимуществам Bitrix24 относятся:

- широкий набор инструментов;
- наличие средств коммуникации внутри системы;
- поддержка корпоративной совместной работы;
- интеграция различных бизнес-процессов в одной платформе.

Недостатками системы являются:

- перегруженность интерфейса;
- высокая сложность освоения;
- избыточность функциональности для задач управления проектами;
- высокие требования к настройке и сопровождению системы.

1.3.6 Сравнительный анализ существующих решений

Проведённый анализ показал, что существующие системы управления проектами обладают широкими функциональными возможностями и позволяют эффективно организовывать работу команд. Однако большинство рассмотренных решений ориентированы либо на крупные корпоративные системы, либо на универсальные платформы с большим количеством дополнительной функциональности.

Для небольших и специализированных систем управления проектами это приводит к ряду недостатков:

- избыточность функциональности;
- сложность интерфейсов;
- высокая трудоёмкость настройки;
- ограниченная адаптация под конкретные бизнес-правила;
- наличие платных ограничений.

Разрабатываемая информационная система ориентирована на более узкую и формализованную модель управления проектами. В отличие от рассмотренных решений, система учитывает строго определённые бизнес-ограничения предметной области, включая:

- ограничение назначения исполнителей только из состава участников проекта;

- контроль завершения проекта при наличии активных задач;
- разграничение ролей пользователей;
- управление состояниями проектов и задач.

Особенностью разрабатываемого решения является то, что система представляет собой API для взаимодействия с данными предметной области. В отличие от рассмотренных платформ, ориентированных преимущественно на готовый пользовательский интерфейс, разрабатываемая система предоставляет централизованный программный интерфейс для работы с проектами, задачами и пользователями.

Такой подход позволяет:

- отделить серверную логику от клиентской части;
- обеспечить возможность интеграции с различными клиентскими приложениями;
- упростить расширение функциональности системы;
- обеспечить централизованную обработку и контроль данных;
- повысить гибкость дальнейшего развития системы.

Кроме того, разрабатываемая система ориентирована на реализацию только необходимой функциональности без избыточных модулей, что позволяет упростить структуру системы и снизить сложность её использования и сопровождения.

1.3.7 Вывод по анализу рынка существующих решений

Проведённый анализ существующих систем управления проектами показал, что современные решения обладают широкими функциональными возможностями и позволяют эффективно организовывать совместную работу участников проектов. Рассмотренные системы поддерживают управление задачами, распределение исполнителей, контроль сроков выполнения и взаимодействие пользователей.

Однако большинство существующих решений ориентированы на универсальное применение и содержат значительное количество

дополнительной функциональности, не всегда необходимой для решения конкретных задач управления проектами. Это приводит к усложнению интерфейсов, повышению трудоёмкости настройки и сопровождения систем, а также увеличению требований к обучению пользователей.

Кроме того, часть рассмотренных решений ориентирована преимущественно на готовые пользовательские интерфейсы и комплексные корпоративные платформы, тогда как разрабатываемая система представляет собой API для централизованного взаимодействия с данными предметной области. Такой подход обеспечивает разделение серверной и клиентской частей системы, упрощает интеграцию с различными приложениями и позволяет более гибко развивать систему в дальнейшем.

Также особенностью разрабатываемого решения является учёт бизнес-ограничений предметной области, связанных с управлением проектами, задачами и ролями пользователей. В отличие от универсальных платформ, система ориентирована на реализацию необходимого набора функций без избыточных модулей и дополнительных компонентов.

Таким образом, разработка специализированной информационной системы управления проектами является актуальной задачей, направленной на создание более структурированного и адаптированного решения для автоматизации процессов проектной деятельности.

1.4 Обоснование и выбор инструментальных средств разработки

1.4.1 Выбор платформы разработки

Одним из наиболее важных этапов разработки информационной системы является выбор технологической платформы. От выбранных инструментов зависят производительность приложения, удобство сопровождения, масштабируемость и возможность дальнейшего развития системы.

Наиболее распространёнными платформами для разработки серверных приложений являются .NET и Java. Обе платформы широко используются при создании корпоративных информационных систем и предоставляют развитые средства для разработки API, работы с базами данных и организации многослойной архитектуры.

В экосистеме Java для разработки серверных приложений чаще всего используется Spring Framework и его модуль Spring Boot. В экосистеме .NET аналогичную роль выполняет ASP.NET Core [1], [2].

Spring Boot предоставляет широкий набор инструментов для создания Web API, организации внедрения зависимостей, работы с базами данных и обеспечения безопасности приложения. Однако для настройки приложения в экосистеме Java зачастую требуется подключение большого количества дополнительных библиотек и конфигураций.

ASP.NET Core, в свою очередь, предоставляет встроенные средства:

- маршрутизации запросов;
- внедрения зависимостей;
- обработки HTTP-запросов;
- аутентификации и авторизации;
- работы с JSON;
- middleware-конвейера;
- конфигурации приложения.

Одним из преимуществ ASP.NET Core является высокая производительность серверных приложений. Для систем, ориентированных на предоставление API, производительность обработки HTTP-запросов является особенно важной характеристикой.

Дополнительным преимуществом платформы .NET является единая экосистема разработки. Большая часть инфраструктурных возможностей уже встроена в платформу и интегрируется между собой без необходимости сложной настройки.

При выборе платформы также важную роль играет используемый язык программирования.

В экосистеме Java основным языком является Java. Для платформы .NET был выбран язык C# 14 [3].

Современные версии C# предоставляют широкий набор языковых возможностей:

- LINQ;
- nullable types;
- pattern matching;
- records;
- generics;
- object-oriented programming;
- строгая типизация;
- улучшенные средства работы с коллекциями;
- встроенная поддержка асинхронного программирования.

Данные возможности позволяют уменьшить количество шаблонного кода и повысить читаемость приложения.

По сравнению с Java язык C# обладает более современным синтаксисом и более тесной интеграцией с инструментами своей платформы .NET.

Разрабатываемая система представляет собой API для взаимодействия с данными предметной области, поэтому при выборе платформы основными критериями являлись:

- производительность серверной части;
- удобство разработки Web API;
- поддержка асинхронной обработки запросов;
- простота конфигурации;
- удобство интеграции компонентов платформы.

В результате сравнения для разработки системы была выбрана платформа .NET 10 с использованием ASP.NET Core 10 и языка программирования C# 14.

1.4.2 Выбор системы управления базами данных

Для хранения данных информационной системы была выбрана система управления базами данных PostgreSQL [4], [5].

Наиболее близкой альтернативой PostgreSQL является Microsoft SQL Server. Обе системы являются реляционными СУБД и поддерживают транзакции, индексацию, механизмы обеспечения целостности данных и выполнение сложных SQL-запросов.

Microsoft SQL Server широко используется в корпоративных приложениях и является частью экосистемы .NET. Однако часть возможностей SQL Server доступна только в коммерческих редакциях.

PostgreSQL представляет собой свободно распространяемую объектно-реляционную СУБД с широким набором встроенных возможностей.

К преимуществам PostgreSQL относятся:

- поддержка пользовательских типов данных;
- встроенная поддержка enum-типов;
- расширенные возможности работы с JSON и массивами;
- высокая надёжность;
- поддержка транзакций;
- кроссплатформенность;
- свободное распространение.

Одним из важных преимуществ PostgreSQL для разрабатываемой системы является встроенная поддержка enum-типов. Использование перечислений позволяет более точно описывать состояния и типы данных предметной области, включая статусы задач, статусы проектов и приоритеты.

Дополнительно PostgreSQL предоставляет расширенные возможности работы со сложными типами данных и поддерживает гибкое расширение функциональности.

Для интеграции PostgreSQL с платформой .NET используется библиотека Npgsql.EntityFrameworkCore.PostgreSQL v10.0.1 [6].

Таким образом, PostgreSQL была выбрана в качестве СУБД благодаря высокой надёжности, поддержке современных типов данных и удобной интеграции с платформой .NET.

1.4.3 Обоснование выбора используемых библиотек и компонентов

Для реализации серверной части системы был выбран набор библиотек и компонентов, обеспечивающих работу с базой данных, организацию бизнес-логики, валидацию данных, аутентификацию пользователей, документирование API и тестирование приложения.

Использование специализированных библиотек позволяет уменьшить объём инфраструктурного кода, повысить читаемость проекта и упростить сопровождение приложения.

1.4.3.1 Entity Framework Core

Для организации взаимодействия с базой данных был выбран Entity Framework Core v10.0.8 [7], [8].

Entity Framework Core представляет собой ORM-фреймворк для платформы .NET, предназначенный для проекции объектов предметной области на структуру реляционной базы данных.

Использование ORM позволяет:

- избежать написание SQL-кода;
- упростить работу с сущностями;
- централизованно управлять моделями данных;
- использовать миграции базы данных;
- повысить удобство сопровождения приложения.

Дополнительно используется библиотека EFCore.NamingConventions v10.0.1, обеспечивающая автоматическое приведение имен таблиц и полей к единому стилю именования.

В экосистеме Java аналогичную роль выполняют Hibernate и Spring Data JPA. Однако Entity Framework Core отличается более простой интеграцией с ASP.NET Core и меньшим количеством конфигурационного кода.

1.4.3.2 MediatR

Для организации взаимодействия между компонентами приложения была выбрана библиотека MediatR v14.1.0 [9].

MediatR реализует паттерн Mediator [10], [11], что позволяет уменьшить связанность компонентов системы. Вместо прямого взаимодействия между сервисами используется механизм запросов и обработчиков.

Использование MediatR позволяет:

- разделить логику приложения по отдельным обработчикам;
- уменьшить связанность компонентов;
- повысить читаемость структуры проекта;
- упростить тестирование;
- улучшить масштабируемость приложения.

В экосистеме Java аналогичная функциональность часто реализуется вручную.

1.4.3.3 FluentValidation

Для проверки корректности входных данных используется библиотека FluentValidation v 12.1.1 [12].

FluentValidation предоставляет декларативный способ описания правил валидации объектов. Использование отдельного слоя валидации позволяет:

- отделить проверку данных от логики обработчиков;
- уменьшить количество проверок внутри обработчиков;
- централизованно управлять правилами валидации;
- повысить читаемость кода.

Преимуществом `FluentValidation` является удобный синтаксис и интеграция с `ASP.NET Core`. При совместном использовании с `MediatR` позволяет реализовать автовалидацию моделей.

Для регистрации в механизме внедрения зависимостей использовалась дополнительная библиотека `FluentValidation.DependencyInjectionExtensions v12.1.1`.

В экосистеме Java аналогичную роль выполняет `Jakarta Bean Validation`.

1.4.3.4 Средства аутентификации и авторизации

Для организации аутентификации и авторизации пользователей используется JWT-аутентификация [13].

JWT представляет собой механизм передачи токенов доступа между клиентом и сервером и широко применяется при разработке Web API.

Для интеграции JWT-аутентификации используется библиотека `Microsoft.AspNetCore.Authentication.JwtBearer v10.0.8`.

Использование JWT позволяет:

- организовать stateless-аутентификацию;
- обеспечить масштабируемость системы;
- разделить серверную и клиентскую части приложения.

Для хеширования паролей используется библиотека `BCrypt.Net-Next v4.2.0`.

1.4.3.5 Средства документирования API

Для документирования API используются `Microsoft.AspNetCore.OpenApi v10.0.8` [14] и `Scalar.AspNetCore v2.14.14` [15].

`OpenAPI` предоставляет стандарт описания Web API и позволяет автоматически формировать документацию на основе серверного приложения.

Scalar используется для отображения интерактивной документации API и тестирования HTTP-запросов.

Использование данных инструментов позволяет упростить тестирование API и повысить удобство интеграции системы.

1.4.3.6 Средства тестирования

Для тестирования приложения используются:

- xunit.v3 v3.2.2;
- Microsoft.NET.Test.Sdk v18.5.1;
- coverlet.collector v10.0.1.

xUnit [16] является современным фреймворком модульного тестирования для платформы .NET.

Использование модульного тестирования позволяет повысить надёжность приложения и упростить сопровождение проекта.

Библиотека coverlet.collector используется для анализа покрытия кода тестами.

В экосистеме Java аналогичную функциональность предоставляют JUnit и JaCoCo.

2 СПЕЦИАЛЬНАЯ ЧАСТЬ

2.1 Проектирование архитектуры системы

2.1.1 Расширение модели предметной области

В процессе проектирования архитектуры системы модель предметной области была расширена дополнительными ролями пользователей, необходимыми для разграничения прав доступа и организации управления системой.

Для управления учетными записями пользователей в системе была добавлена роль администратора. Пользователь с данной ролью обладает возможностью создания, изменения и удаления пользователей, а также управления ролями учетных записей.

Дополнительно была введена роль руководителя. Руководитель обладает полномочиями администратора, менеджера и исполнителя, а также имеет возможность назначать менеджеров на проекты.

Таким образом, в системе были выделены следующие роли пользователей:

- исполнитель;
- менеджер проекта;
- администратор;
- руководитель.

Использование ролей позволяет реализовать разграничение доступа к функциональности системы и ограничить выполнение операций в зависимости от полномочий пользователя.

2.1.2 Ограничения и бизнес-правила системы

При проектировании системы были определены ограничения и бизнес-правила, обеспечивающие корректность работы предметной области и целостность данных.

1) Для управления пользователями определены следующие ограничения:

- а) администратор отвечает за управление учетными записями пользователей;
- б) исполнитель может работать только с назначенными ему задачами;
- в) менеджер управляет только назначенными ему проектами;
- г) руководитель обладает полным доступом к функциональности системы.

2) Для сущности проекта были определены следующие бизнес-правила:

- а) проект содержит пул исполнителей;
- б) менеджер проекта не входит в состав исполнителей;
- в) проект может не иметь назначенного менеджера;
- г) проект может управляться руководителем без участия менеджера;
- д) завершённый проект запрещено изменять, исключением является удаление пользователей из проекта;
- е) приоритет проекта не может быть изменён после создания;
- ж) завершение проекта невозможно при наличии незавершённых задач.

3) Для сущности задачи были определены следующие ограничения:

- а) задача принадлежит только одному проекту;
- б) исполнитель задачи назначается только из состава исполнителей проекта;
- в) одна задача может иметь только одного исполнителя;
- г) задача может существовать без исполнителя;
- д) завершение задачи невозможно без назначения исполнителя;

- е) завершённую задачу запрещено изменять, исключением является удаление связей с пользователями.
- 4) Дополнительно были определены правила удаления связанных данных.
 - а) при удалении проекта автоматически удаляются связанные задачи и состав исполнителей проекта;
 - б) при удалении пользователя, который назначен исполнителем задачи, ссылка на него изменяется на null;
 - в) при удалении пользователя, который входит в состав исполнителей проекта, он удаляется из состава проекта;
 - г) при удалении пользователя, который является менеджером проекта, ссылка на менеджера изменяется на null.
- 5) Для описания состояний предметной области используются перечисления:
 - а) ProjectPriority: Low, Medium, High;
 - б) ProjectStatus: Active, Completed;
 - в) WorkTaskStatus: Active, Completed.

Использование бизнес-правил и ограничений позволяет обеспечить корректное состояние объектов предметной области и предотвратить выполнение недопустимых операций.

2.1.3 Выбор архитектурного подхода

При проектировании системы была выбрана архитектура, ориентированная на построение Web API с разделением ответственности между слоями приложения.

Основными требованиями к архитектуре являлись:

- разделение бизнес-логики и инфраструктурного кода;
- централизованная обработка запросов;
- удобство расширения функциональности;
- поддержка тестируемости компонентов;

- минимизация избыточных абстракций;
- сохранение относительно простой структуры проекта.

В качестве основного архитектурного подхода была выбрана Vertical Slice Architecture (VSA) [17].

В отличие от классической многослойной архитектуры, где структура приложения строится вокруг технических слоев, VSA ориентируется на функциональные возможности системы.

Использование Vertical Slice Architecture позволяет:

- уменьшить связанность компонентов;
- упростить навигацию по проекту;
- группировать код по функциональности;
- сократить количество инфраструктурного кода;
- повысить удобство сопровождения системы.

Для реализации API используется Minimal API.

Minimal API представляет собой упрощённый подход к построению HTTP API в ASP.NET Core, позволяющий сократить объём шаблонного кода и отказаться от использования контроллеров.

Использование Minimal API позволяет:

- уменьшить количество инфраструктурного кода;
- упростить регистрацию маршрутов;
- сократить количество промежуточных абстракций;
- повысить читаемость API.

2.1.4 Использование CQRS-lite и Mediator

Для организации обработки запросов в системе используется упрощённый вариант архитектурного подхода CQRS (Command Query Responsibility Segregation).

Классический CQRS предполагает разделение операций чтения и изменения данных, а также использование отдельных моделей для команд и запросов.

В разрабатываемой системе используется облегчённый вариант CQRS (CQRS-lite), при котором:

- команды и запросы разделяются логически;
- используются отдельные обработчики для операций чтения и изменения;
- используется единая база данных.

Использование CQRS-lite позволяет сохранить преимущества разделения ответственности без существенного усложнения архитектуры.

Для организации взаимодействия между компонентами приложения используется библиотека MediatR, реализующая паттерн Mediator.

Mediator обеспечивает централизованную передачу запросов между компонентами системы и уменьшает связанность между слоями приложения.

При использовании MediatR HTTP-запрос сначала поступает в endpoint Minimal API, после чего передаётся в Mediator. Далее Mediator определяет соответствующий обработчик и передаёт управление ему.

Обработчик выполняет программную и бизнес логику, взаимодействует с базой данных и возвращает результат выполнения операции.

Использование Mediator позволяет:

- уменьшить связанность компонентов;
- централизовать обработку запросов;
- упростить тестирование;
- разделить обработку команд и запросов;
- повысить читаемость структуры проекта.

Диаграмма взаимодействия компонентов при использовании Mediator представлена на Рисунке 2.1.

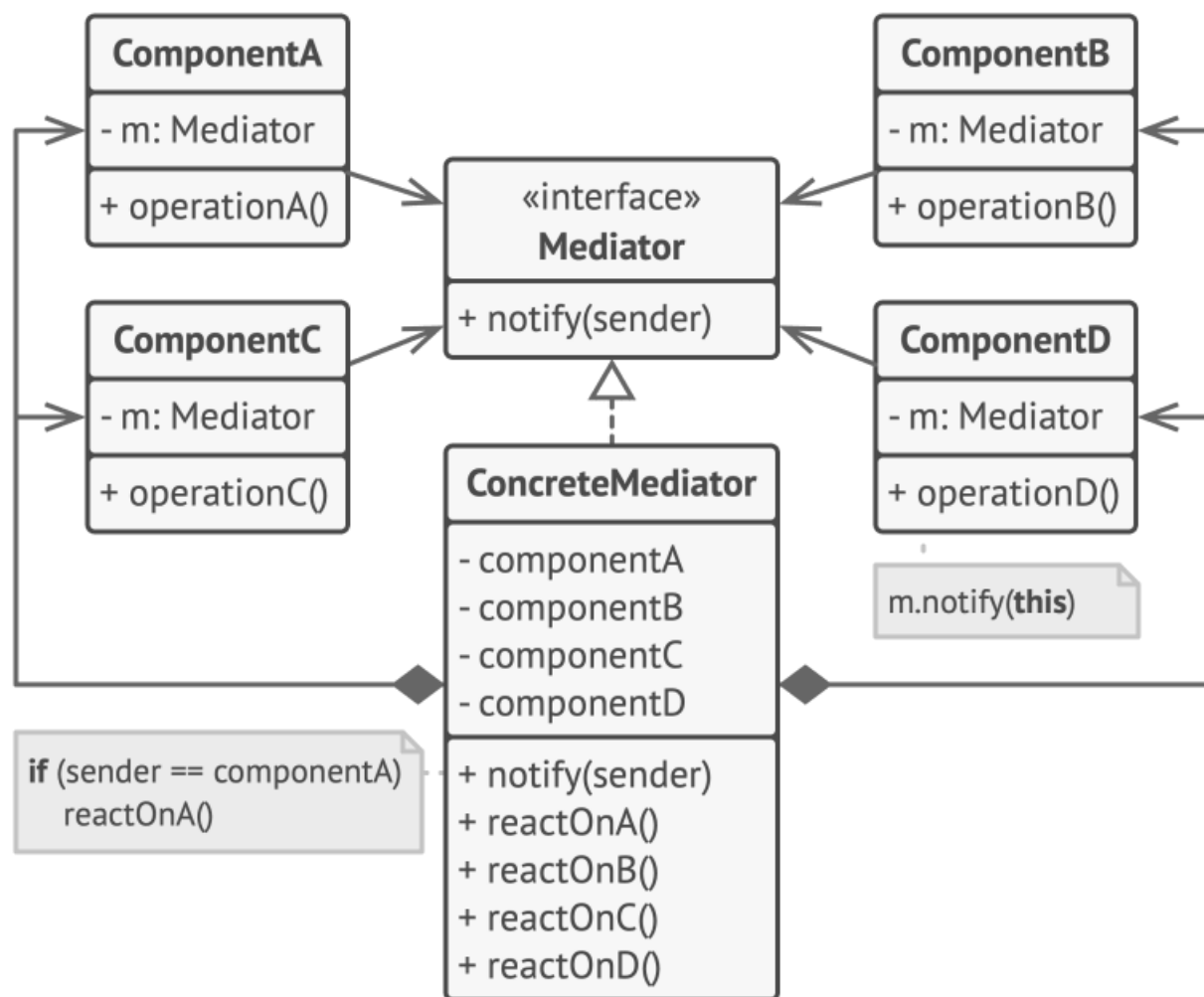


Рисунок 2.1 — Пример реализации паттерна mediator

2.1.5 Использование подхода DDD-lite

При проектировании предметной области использовались отдельные элементы Domain-Driven Design (DDD).

Полноценная реализация DDD предполагает использование большого количества инфраструктурных абстракций, репозитория и отдельных доменных моделей. Для разрабатываемой системы такой подход является избыточным, поэтому был использован упрощённый вариант — DDD-lite.

Основной целью использования DDD-lite является сохранение бизнес-правил внутри сущностей предметной области и уменьшение вероятности нарушения инвариантов системы.

В системе были выделены следующие агрегаты:

- Project — корень агрегата;

- WorkTask — дочерняя сущность проекта;
- ProjectExecutor — связующая сущность между проектом и пользователем;
- User — независимый агрегат.

Корень агрегата Project отвечает за управление задачами и составом исполнителей проекта.

Сущность WorkTask не может существовать вне проекта и управляется через агрегат Project.

Связи между сущностями представлены на Рисунке 2.2.

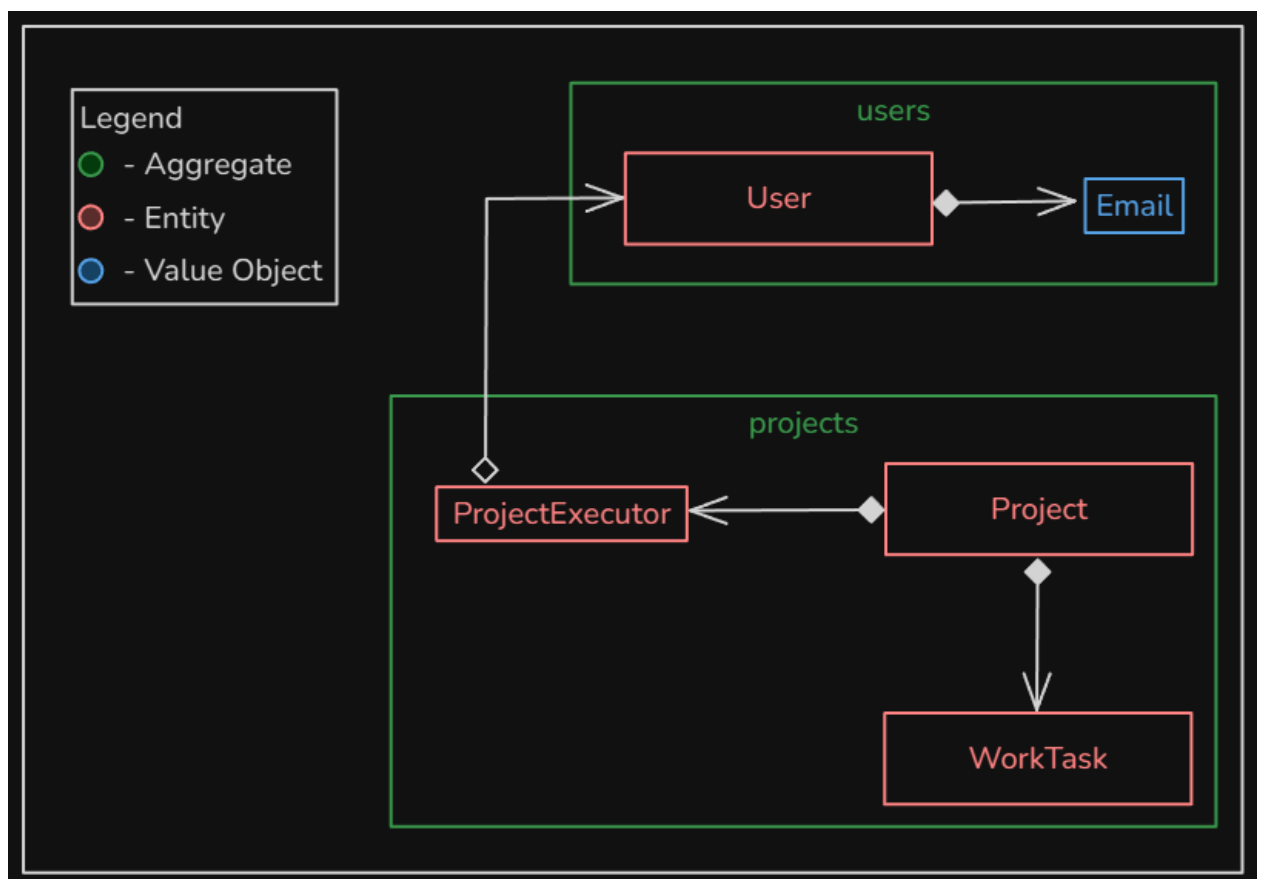


Рисунок 2.2 — Class UML диаграмма с использованием цветowych обозначений

Для реализации доменной модели использовались следующие подходы:

- ограничение прямого изменения состояния сущностей;
- использование методов предметной области;
- использование private set;
- использование private constructor;
- инкапсуляция бизнес-правил внутри сущностей.

При этом архитектура была адаптирована под особенности Entity Framework Core.

Вместо построения отдельного слоя репозитория используется прямое взаимодействие обработчиков с DbContext.

Использование DbContext без дополнительных абстракций позволяет:

- уменьшить количество шаблонного кода;
- избежать дублирования методов ORM;
- упростить сопровождение приложения;
- сократить сложность архитектуры.

2.1.6 Структура слоев приложения

Архитектура приложения разделена на несколько логических слоев.

- 1) Слой Domain содержит:
 - а) сущности предметной области;
 - б) перечисления;
 - в) value object;
 - г) бизнес-правила;
 - д) доменные методы.
- 2) Слой Infrastructure отвечает за:
 - а) взаимодействие с базой данных;
 - б) конфигурацию Entity Framework Core;
 - в) DbContext;
 - г) миграции базы данных.
- 3) Слой API содержит:
 - а) endpoints Minimal API;
 - б) обработчики MediatR;
 - в) DTO;
 - г) валидацию запросов;
 - д) конфигурацию приложения.

Зависимости между слоями организованы следующим образом:

API зависит от Domain и Infrastructure;

Infrastructure зависит от Domain;

Domain не зависит от других слоев.

Такой подход позволяет сохранить независимость доменной модели и разделить бизнес-логику и инфраструктурный код.

Структура зависимостей слоев представлена на Рисунке 2.3.

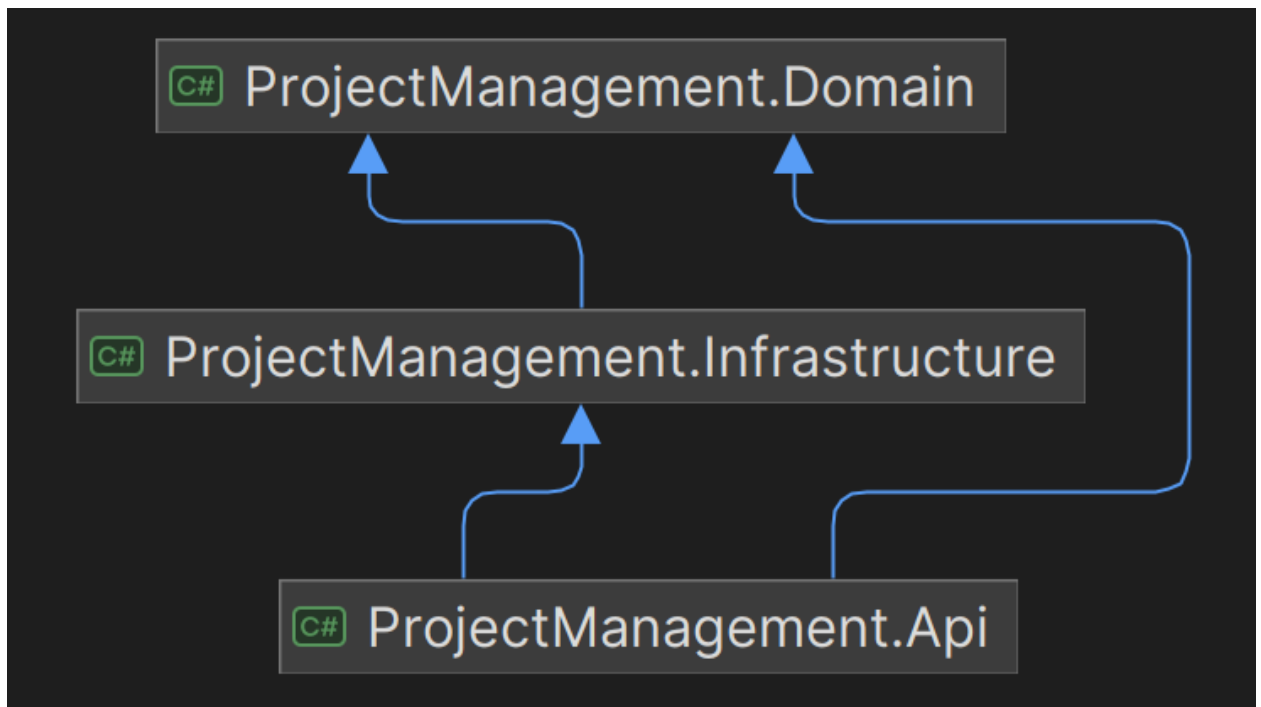


Рисунок 2.3 — Диаграмма зависимостей слоев приложения

2.1.7 Итоги проектирования архитектуры системы

В результате проектирования архитектуры была сформирована структура приложения, ориентированная на разработку масштабируемого и поддерживаемого Web API.

Использование Vertical Slice Architecture, Minimal API, CQRS-lite и MediatR позволило уменьшить связанность компонентов, упростить структуру проекта и организовать централизованную обработку запросов.

Применение элементов DDD обеспечило сохранение бизнес-правил внутри предметной области и позволило реализовать ограничения системы на уровне доменной модели.

Разделение приложения на слои Domain, Infrastructure и API обеспечивает независимость бизнес-логики от инфраструктурных компонентов и упрощает дальнейшее развитие системы.

2.2 Проектирование базы данных

2.2.1 Выбор подхода к проектированию базы данных

При проектировании базы данных был выбран подход Code First, реализованный с использованием ORM-фреймворка Entity Framework Core.

Данный подход предполагает, что структура базы данных формируется на основе классов предметной области, а физическая схема создаётся автоматически на основе определений сущностей и их конфигурации. В отличие от подхода Database First, где сначала создаётся структура базы данных, а затем генерируются классы моделей, Code First позволяет использовать доменную модель как единственный источник истины.

Использование Code First в разрабатываемой системе обусловлено тесной связью доменной модели с бизнес-логикой приложения. Так как система построена на принципах DDD-lite и Vertical Slice Architecture, сущности предметной области уже содержат бизнес-ограничения и поведение, что делает их естественной основой для генерации структуры базы данных.

Дополнительно применялся механизм миграций Entity Framework Core, позволяющий управлять изменениями структуры базы данных на протяжении жизненного цикла системы. Каждое изменение доменной модели сопровождается созданием миграции, что обеспечивает синхронизацию между кодом приложения и структурой базы данных.

Преимуществами выбранного подхода являются:

- описание структуры БД с помощью C#
- единый источник описания структуры данных в виде кода;
- синхронизация доменной модели и базы данных;
- автоматизация создания и изменения схемы БД;

- удобство сопровождения и расширения системы;
- возможность версионирования изменений через миграции;
- снижение риска рассинхронизации между кодом и базой данных.

Использование Code First особенно эффективно в условиях активно развивающейся системы, где структура предметной области может изменяться в процессе разработки.

2.2.2 Логическая модель данных

При проектировании базы данных сущности предметной области были реализованы в виде таблиц реляционной базы данных следующим образом:

- Project : таблица Projects;
- WorkTask : таблица WorkTasks;
- User : таблица Users;
- ProjectExecutor : таблица ProjectExecutors.

Между таблицами были реализованы следующие связи:

- между Projects и WorkTasks реализована связь «один ко многим» (one-to-many), где один проект может содержать множество задач, а задача принадлежит только одному проекту;

- между Users и Projects реализована связь «один ко многим», где один пользователь может быть менеджером нескольких проектов, а проект может иметь только одного менеджера;

- между Users и WorkTasks реализована связь «один ко многим», где один пользователь может быть назначен исполнителем нескольких задач, а задача может иметь только одного исполнителя;

- между Projects и Users реализована связь «многие ко многим» (many-to-many) через промежуточную таблицу ProjectExecutors, определяющую состав исполнителей проекта.

Для отдельных связей используются nullable-внешние ключи:

- Project.ManagerId может содержать null, так как проект может существовать без менеджера;

— WorkTask.ExecutorId может содержать null, так как задача может существовать без назначенного исполнителя.

На Рисунке 2.4 представлена логическая модель БД.

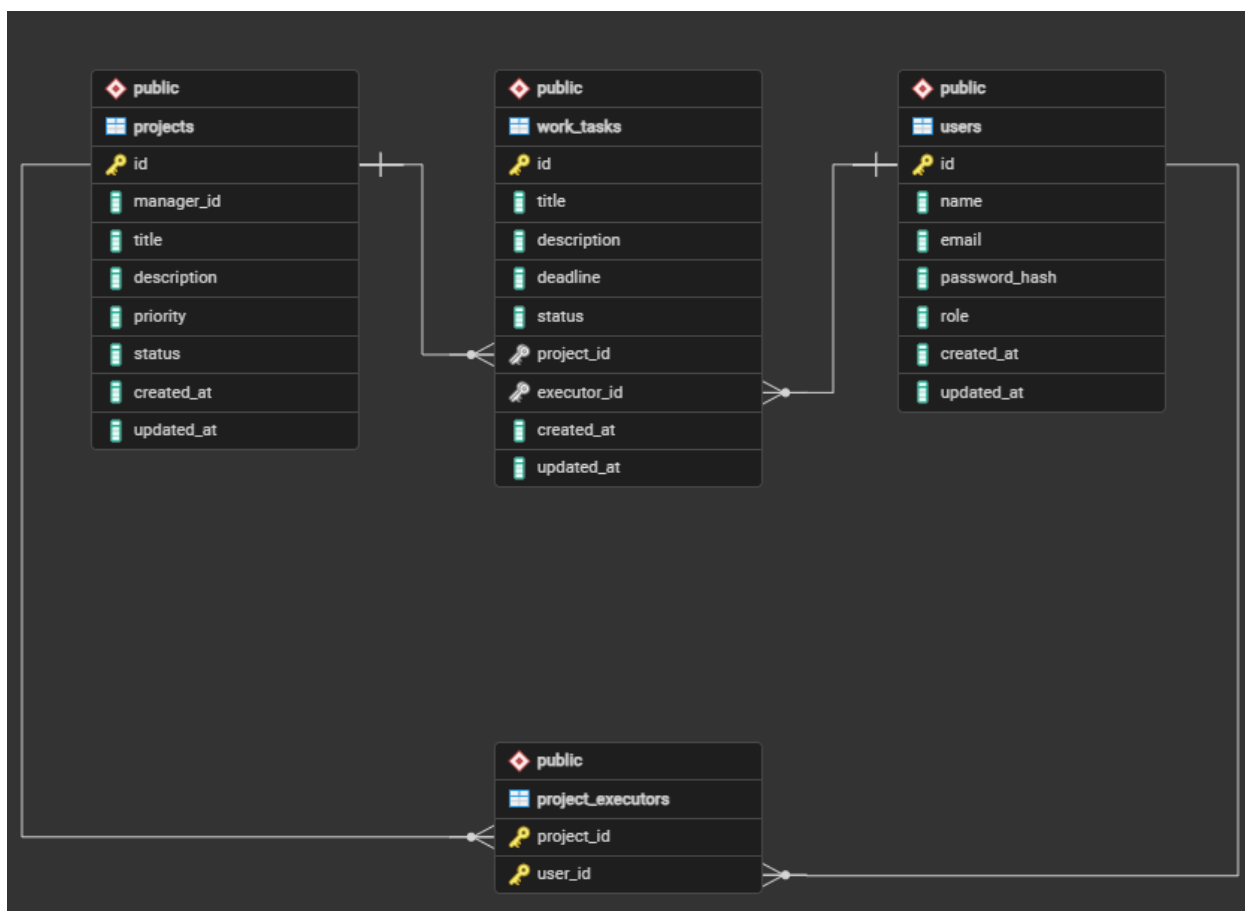


Рисунок 2.4 — Логическая модель БД

2.2.3 Физическая модель базы данных

Физическая модель базы данных реализована на основе выбранной системы управления базами данных PostgreSQL. На данном уровне проектирования определяются конкретные типы данных и правила обеспечения целостности данных.

Для обеспечения целостности данных используются следующие правила:

— каскадное удаление (ON DELETE CASCADE) применяется для удаления задач и записей ProjectExecutors при удалении проекта;

— установка значения NULL (ON DELETE SET NULL) применяется при удалении пользователя, если он является менеджером проекта или исполнителем задачи.

Типы данных в PostgreSQL подбирались с учётом особенностей предметной области, включая использование перечислений для хранения значений статусов и приоритетов:

- UserRole
- ProjectPriority;
- ProjectStatus;
- WorkTaskStatus.

Использование PostgreSQL позволяет эффективно применять ограничения целостности и строго типизированные структуры данных, включая перечисления, что повышает надёжность хранения информации и снижает вероятность неконсистентных состояний в базе данных.

На Рисунке 2.5 представлена физическая модель БД.

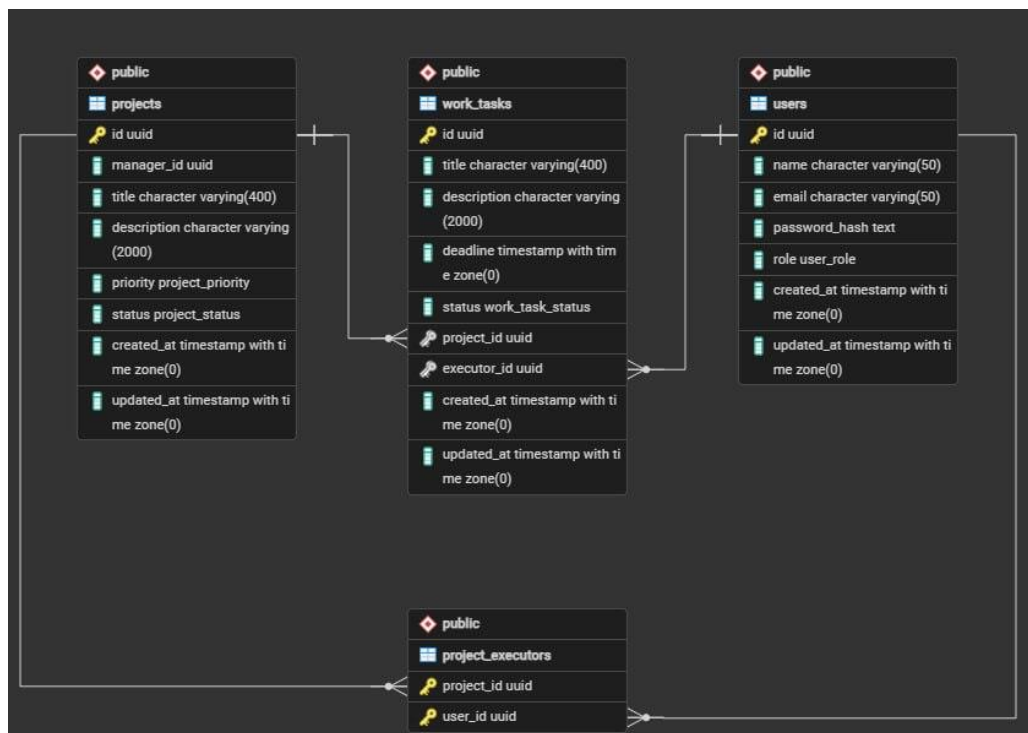


Рисунок 2.5 — Физическая модель PostgreSQL

Код моделей указан в Приложении А.

Код Entity Framework Core конфигураций указан в Приложении Б.

2.3 Разработка серверной части

2.3.1 Реализация Web API с использованием VSA

В соответствии с Vertical Slice Architecture каждая функциональность системы реализуется как отдельная feature (функция), представляющая вертикальный срез (Рисунок 2.6), содержащий обработку запроса (слой Api), бизнес-логику (слой Domain) и взаимодействие с базой данных (слой Infrastructure).

Срез состоит из Endpoint (конечной точки API), Handler (обработчика), а также DTO-объектов, представляющих Command или Query в соответствии с подходом CQRS. Дополнительно в состав среза могут входить валидаторы, DTO ответов и другие вспомогательные компоненты, необходимые для реализации функциональности.

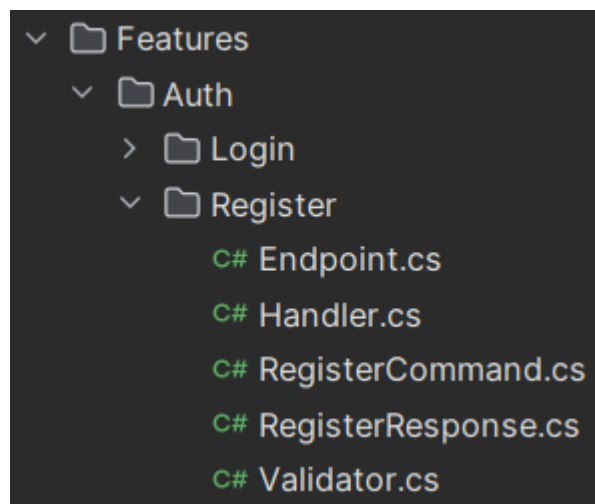


Рисунок 2.6 — Срез для функции "Register"

Данная реализация демонстрирует применение библиотеки MediatR для организации взаимодействия между компонентами системы, а также использование подхода CQRS, обеспечивающего разделение операций чтения (Query) и изменения состояния системы (Command). В соответствии с представленным примером реализована каждая функциональность сервера.

Код реализации данного среза представлен в Приложении В.

2.3.2 Реализация DDD-lite

При разработке серверной части системы использовался подход DDD-lite, предполагающий выделение доменных сущностей и инкапсуляцию бизнес-логики в рамках предметной области без полного внедрения всех паттернов Domain-Driven Design.

В качестве доменных сущностей были реализованы модели пользователей, проектов, задач и связей между ними. Модели содержат основные свойства предметной области, а также навигационные связи, используемые Entity Framework Core для взаимодействия с базой данных.

Код доменных моделей приведён в Приложении А.

2.3.3 Реализация ролевой модели доступа

Для обеспечения безопасности доступа к ресурсам системы были реализованы механизмы идентификации, аутентификации и авторизации пользователей.

На этапе идентификации пользователь передаёт системе свои учетные данные, такие как адрес электронной почты. Далее выполняется аутентификация пользователя путём проверки корректности предоставленного пароля. После успешной аутентификации сервер формирует JWT-токен (JSON Web Token), содержащий идентификатор пользователя и сведения о его роли.

Полученный токен должен использоваться клиентским приложением при выполнении последующих запросов к API для подтверждения личности пользователя. Проверка токена и определение прав доступа выполняются средствами ASP.NET Core Authorization с политик авторизации.

Авторизация обеспечивает разграничение доступа к ресурсам системы в зависимости от роли пользователя.

Код реализации аутентификации и авторизации приведён в Приложении Г.

2.4 Разработка интерфейса

2.4.1 Использование HTTP-методов, REST и RPC

В рамках разработки интерфейса взаимодействия с системой был определён единый подход к организации HTTP [18] API, основанный на комбинировании архитектурных стилей REST (Representational State Transfer) [19] и RPC (Remote Procedure Call) [20]. Данный подход обеспечивает разделение операций чтения и изменения данных и упрощает понимание назначения конечных точек API.

Операции получения данных реализованы в соответствии с принципами REST. Для них используется HTTP-метод GET, который применяется для получения ресурсов системы, таких как пользователи, проекты и задачи. Идентификаторы сущностей передаются через параметры маршрута, а дополнительные параметры фильтрации и поиска — через query-параметры строки запроса. Тело запроса (body) в GET-запросах не используется. Данный тип запросов соответствует Query-операциям в архитектуре CQRS.

Пример REST-запроса: *GET /projects/{id}*

Операции изменения состояния системы реализованы в соответствии с RPC-подходом поверх HTTP-протокола. Для них используется метод POST, а выполняемое действие явно выражается в маршруте запроса. Идентификаторы сущностей передаются через параметры маршрута, а остальные данные — в теле запроса. Данный тип операций соответствует Command-операциям в CQRS.

Пример RPC-запроса: *POST /users/{id}/change-email*

Отдельно выделяются операции удаления, которые реализованы с использованием HTTP-метода DELETE. В рамках REST-подхода идентификатор удаляемой сущности передаётся исключительно через параметр маршрута. Данные операции также относятся к Command-операциям, однако рассматриваются как специальный случай изменения состояния системы.

Пример запроса удаления: DELETE /users/{id}

Все операции, возвращающие результат выполнения, отдают данные в формате JSON с HTTP-статусом 200 OK. В случае выполнения операций, не возвращающих данных (например, удаление или команды без результата), используется статус 204 No Content.

2.4.2 Установленные конвенции интерфейса API

В рамках проектирования интерфейса взаимодействия с системой были установлены следующие соглашения:

- 1) GET-запросы выполняются в соответствии с REST-подходом и используются исключительно для получения данных (Query-операции CQRS).
 - а) Идентификаторы передаются через параметры маршрута.
 - б) Параметры фильтрации и поиска передаются через query string.
 - в) Тело запроса не используется.
 - г) POST-запросы (Command-операции) реализуются в соответствии с RPC-подходом.
 - д) Имя действия явно отражается в маршруте.
 - е) Идентификаторы сущностей передаются через параметры маршрута.
 - ж) Остальные данные передаются в теле запроса.
- 2) DELETE-запросы соответствуют REST-подходу и используются для удаления сущностей.
 - а) Идентификатор передаётся только через маршрут.
 - б) Рассматриваются как Command-операции специального типа.
- 3) Все запросы, возвращающие данные, возвращают тело ответа в формате JSON с кодом 200 OK.
- 4) Запросы без возвращаемого результата возвращают статус 204 No Content.

2.4.3 DTO-модели для обмена данными

В рамках проектирования интерфейса взаимодействия между клиентом и сервером были использованы DTO-модели (Data Transfer Objects), предназначенные для обмена данными через HTTP API. Их применение позволяет отделить внутренние доменные модели системы от внешнего контракта API и обеспечить стабильность интерфейса при изменении бизнес-логики.

DTO используются для передачи данных между клиентской и серверной частью системы и разделяются на модели запросов и ответов (Response).

Модели запросов в рамках CQRS-подхода дополнительно классифицируются:

- Query — используются для операций чтения данных (обычно HTTP GET);
- Command — используются для операций изменения состояния системы (POST, PUT, PATCH, DELETE);
- Request — вспомогательные DTO, применяемые в случаях, когда входные данные приходят из нескольких источников (например, из маршрута и тела запроса одновременно) и требуют композиции перед формированием Command или Query.

Таким образом, Command и Query представляют прикладные операции, обрабатываемые на уровне бизнес-логики, тогда как Request служат лишь промежуточной моделью для удобства связывания входных данных API.

2.4.4 Документирование API

Для документирования HTTP API в системе используется спецификация OpenAPI, которая позволяет формализовать описание всех доступных конечных точек, их параметров, моделей данных и возможных ответов.

На основе OpenAPI-описания автоматически формируется интерактивная документация с использованием инструмента Scalar (Рисунок 2.7), предоставляющего удобный интерфейс для взаимодействия с API.

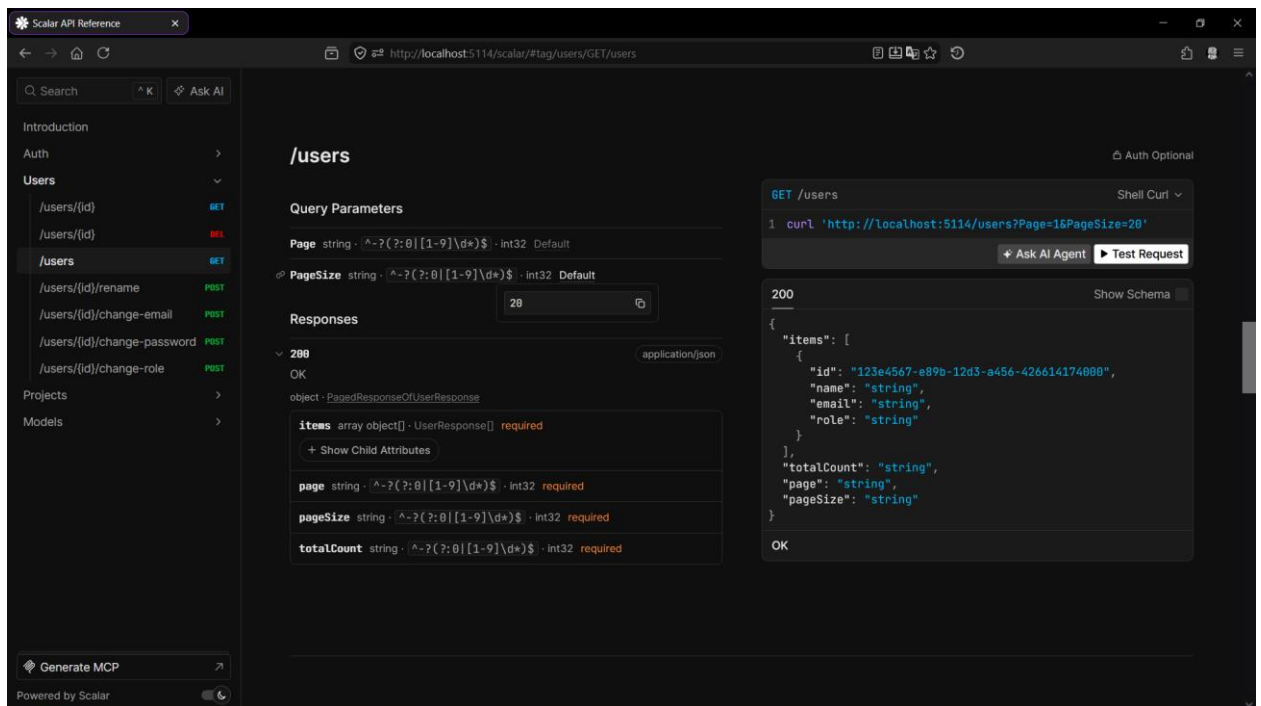


Рисунок 2.7 — Внешний вид интерактивной документации

Использование автоматической генерации документации обеспечивает актуальность описания интерфейса при изменениях в кодовой базе и снижает вероятность расхождения между реализацией и документацией.

Документация API предоставляет следующие возможности:

- просмотр списка доступных конечных точек и их HTTP методов;
- тестирование HTTP-запросов непосредственно из интерфейса документации;
- отображение схем запросов и ответов;
- описание кодов ответов и форматов данных.

Таким образом, документирование API обеспечивает удобство разработки, тестирования и интеграции между клиентскими приложениями и серверной частью системы.

2.5 Тестирование и отладка программного продукта

2.5.1 Модульное тестирование

Тестирование программного продукта проводилось с целью проверки корректности работы реализованной бизнес-логики доменного слоя системы. Основное внимание уделялось проверке корректности выполнения бизнес-правил, обработке исключительных ситуаций и обеспечению стабильности работы компонентов предметной области.

Для проверки работоспособности доменного слоя были разработаны автоматизированные модульные тесты с использованием фреймворка xUnit. Тестирование включает проверку:

- управления проектами и задачами;
- корректности изменения состояния доменных сущностей;
- соблюдения бизнес-ограничений и правил валидации;
- обработки исключительных ситуаций.

Разработанные тесты обеспечивают покрытие основной бизнес-логики системы и позволяют выявлять ошибки на ранних этапах разработки, а также контролировать корректность работы приложения при внесении изменений в программный код.

В процессе отладки использовались встроенные средства среды разработки и механизмы логирования ASP.NET Core, позволяющие анализировать выполнение программного кода и выявлять причины возникновения ошибок.

На рисунке 2.8 показаны успешные результаты тестов.

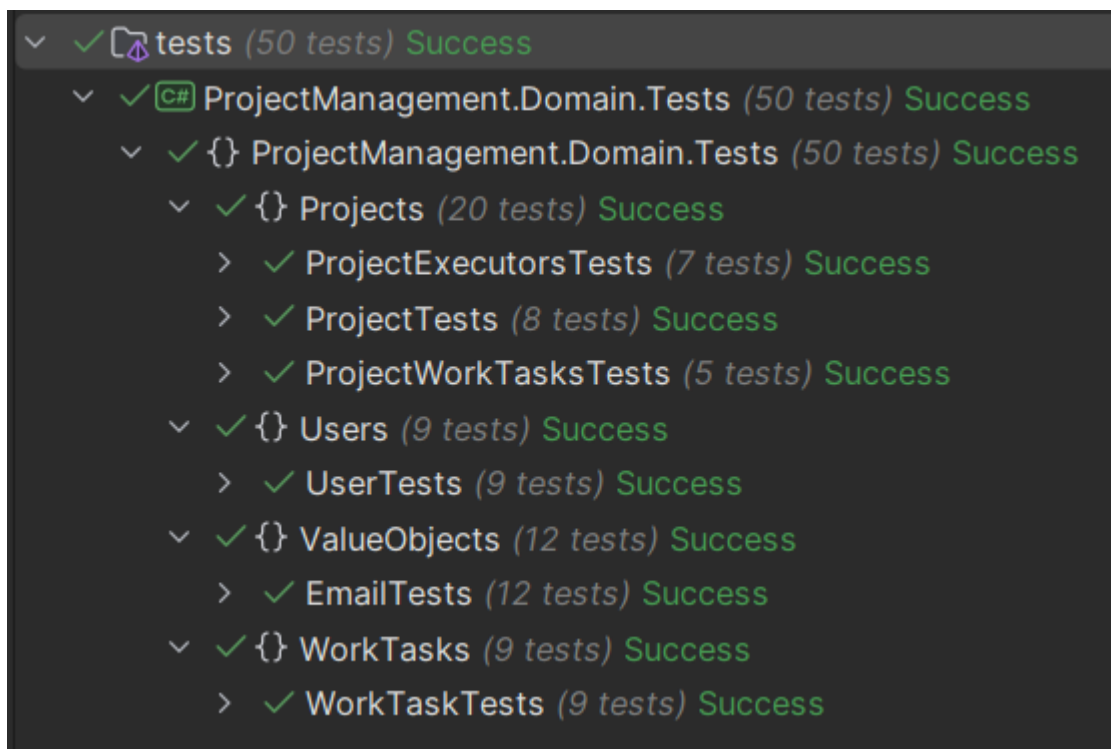


Рисунок 2.8 — Результаты тестов

Код реализованных тестов представлен в Приложении Д.

2.5.2 Проверка интерфейса интерактивной документации Scalar

Дополнительно была выполнена визуальная проверка интерактивной документации API, реализованной с использованием Scalar. В ходе проверки оценивалась корректность отображения маршрутов API, описаний методов, параметров запросов и моделей данных.

Также проверялась работоспособность встроенных средств отправки HTTP-запросов, корректность отображения кодов ответа сервера и структуры возвращаемых данных. Проверка позволила убедиться в корректной генерации OpenAPI-спецификации и удобстве взаимодействия с API через интерактивный интерфейс документации.

На Рисунке 2.9 показано корректное определение маршрута, тела запроса, типа и кода ответа.

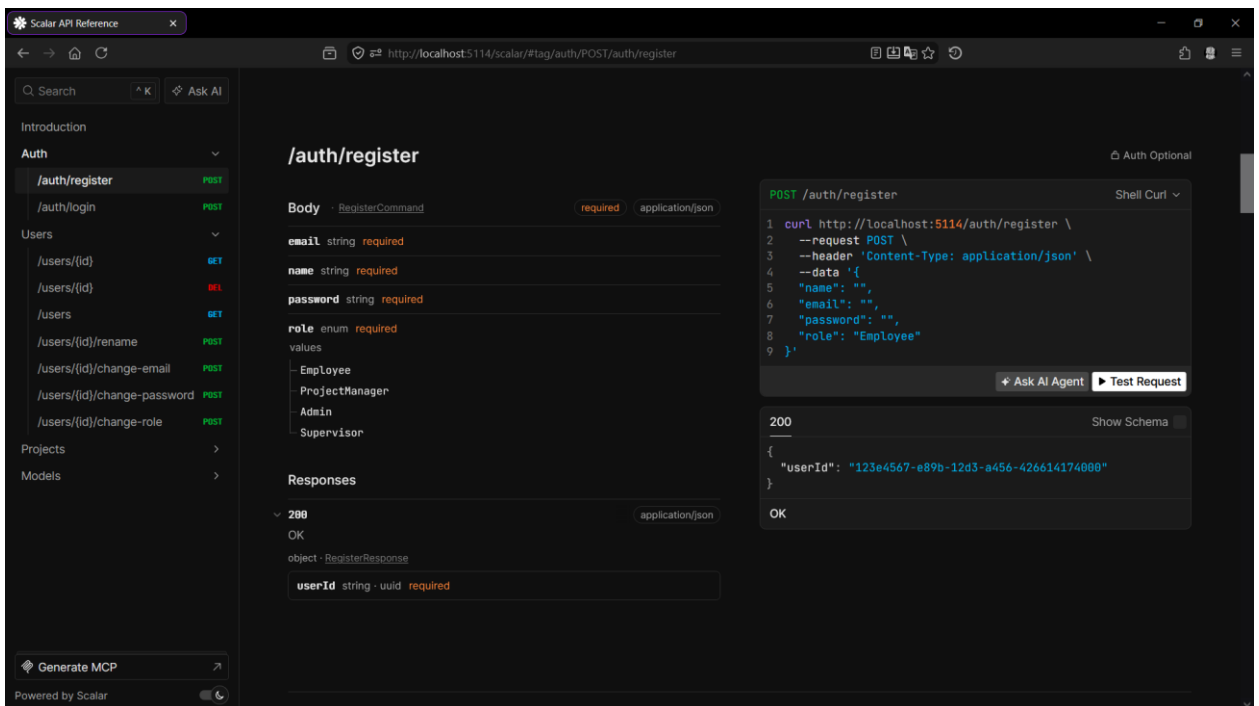


Рисунок 2.9 — Полное описание конечной точки регистрации

На Рисунках 2.10-2.11 показано успешное выполнение запроса на регистрацию.

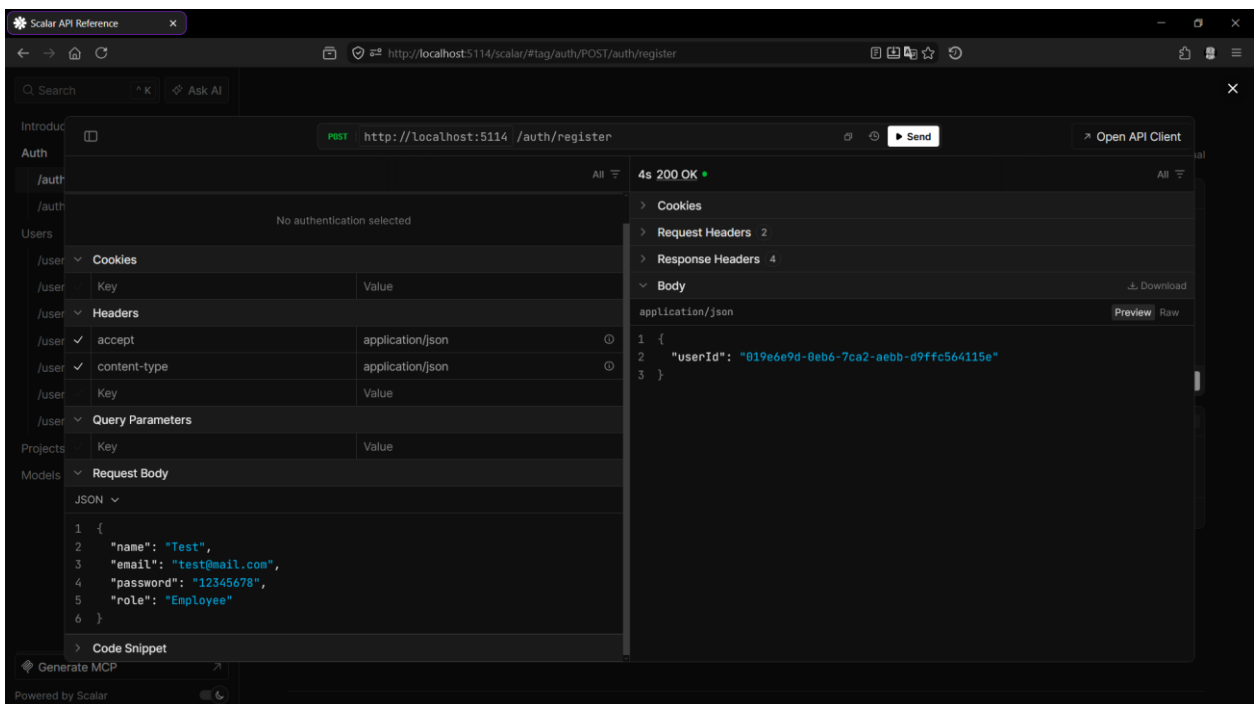


Рисунок 2.10 — Выполнение запроса на регистрацию

Слева снизу отображается JSON, представляющий тело запроса, значения которого нужно задать вручную. А с правой стороны отображается тело и статусный код ответа.

id	name	email	password_hash	role	created_at	updated_at
019e6e9d-0eb6-7ca2-aebb-d9ffc564115e	Test	test@mail.com	\$2a\$11\$LABD0ZRxLHisvMmmzgVH0iQ2LSRNWuk3.tDV...	employee	2026-05-28 12:44:07 +00:00	2026-05-28 12:44:07 +00:00

Рисунок 2.11 — Запись только что зарегистрированного пользователя в БД

Запрос выполнялся успешно, и по этой причине была создана новая запись в БД. По id можно убедиться, что это только что созданный пользователь.

На Рисунке 2.12 отображается факт успешного автодокументирования моделей системы и их ограничений.

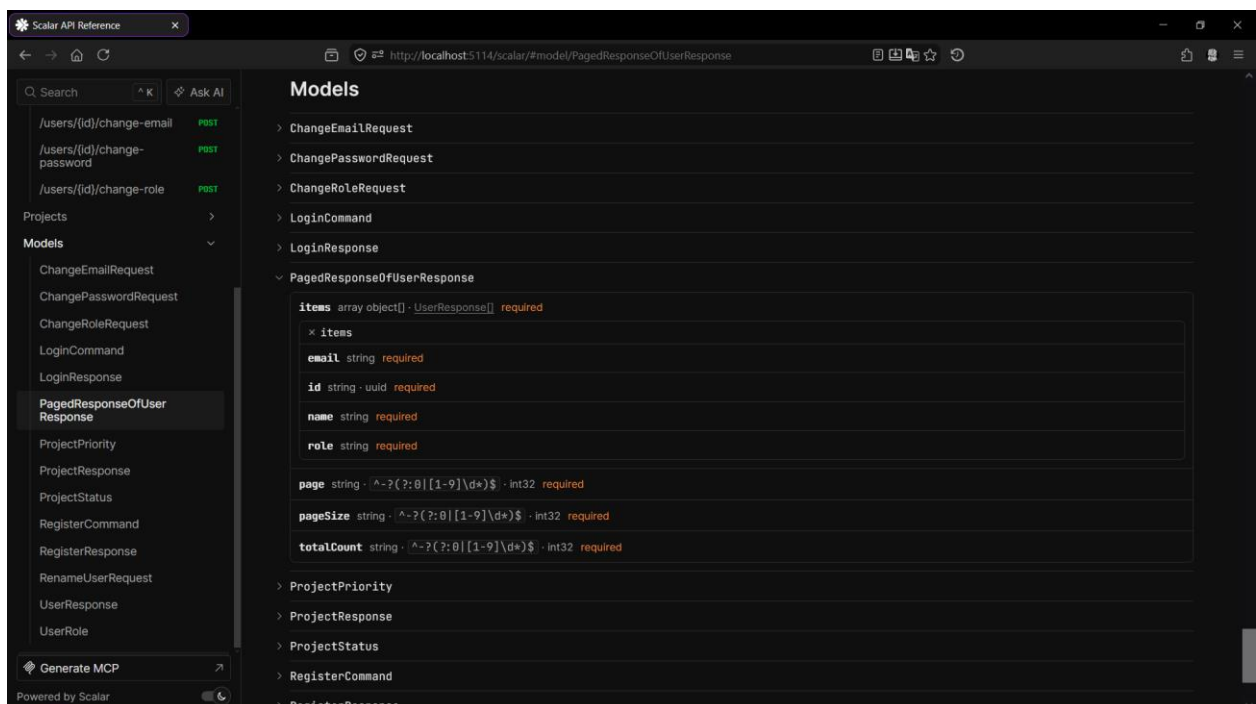


Рисунок 2.12 — Модели системы и их описание

3 ЭКОНОМИЧЕСКАЯ ЧАСТЬ

3.1 Область применения программного продукта и его преимущества перед аналогичным программным продуктом

Разработанное приложение для предприятия, предназначено для организации централизованной автоматизированной системы управления проектами.

Достоинствами приложения являются система обеспечивающую централизованное хранение информации о проектах, задачах и пользователях, автоматизацию бизнес-правил и разграничение прав доступа в зависимости от роли пользователя.

3.2 Трудоемкость разработки программного продукта, квалификация исполнителя и его оклад

Трудоемкость разработки можно определить в Таблице 3.1. Строка «Всего» отображает общую трудоемкость разработки.

Таблица 3.1 — Трудоемкость разработки программного продукта

Наименование этапа	Условное обозначение	Трудоемкость выполнения этапа, ч.
Описание технического задания	Ттз	6
Проектирование приложения	Тпс	20
Разработка серверной стороны	Тбд	75
Разработка приложения	Трс	125
Отладка приложения	Тос	30
Тестирование приложения	Ттс	20
Оформление документации	Тод	20
Всего	Тобщ	296

Оклад и тарифная ставка разработчиков программного продукта представлены в Таблице 3.2.

Таблица 3.2 — Разработчики программного продукта

Исполнители	Оклад, руб.	Часовая тарифная ставка, руб./ч.	Количество сотрудников
Разработчик-программист	100 000	625	1
Руководитель	145 000	906,25	1

Часовая тарифная ставка ЧТС, руб./ч., определяется исходя из месячного оклада, количества рабочих дней в месяце и продолжительности рабочего дня по формуле:

$$\text{ЧТС} = \frac{\text{Ом}}{\text{Д} \times \text{Тс}}, \text{ руб./ч.}, \quad (3.1)$$

где Ом — оклад исполнителя в месяц, руб./мес.;

Д — количество рабочих дней в месяце (для расчета Д = 20 раб. дней);

Тс — продолжительность рабочего дня (для расчета Тс = 8 ч.).

$$\text{ЧТС1} = \frac{100000}{20 \times 8} = \frac{100000}{160} = 625,00, \text{ руб./ч.}, \quad (3.2)$$

$$\text{ЧТС2} = \frac{145000}{20 \times 8} = \frac{145000}{160} = 906,25, \text{ руб./ч.}, \quad (3.3)$$

В Таблице 3.3 показана стоимость технических средств для разработки системы.

Таблица 3.3 — Стоимость технических средств разработки

Наименование компонента	Цена, руб.	Количество, шт.	Стоимость, руб.
15.6" Ноутбук MSI Modern 15 D7M-217XRU	76 999	1	76 999
Мышь беспроводная Red Square	2 999	1	2 999
Принтер лазерный Canon image Class LBR6030	14 599	1	14 599
Компьютерное кресло офисное Domfy CP-X	3034	1	3 034
Итого:			97 631

В Таблице 3.4 представлены затраты на расходные материалы.

Таблица 3.4 — Планируемые затраты на расходные материалы

Затраты	Стоимость	Количество	Сумма, руб.
Интернет	749 руб./мес.	2 мес.	1 200
Электричество	7,08 руб./КВт*ч	310 КВт*ч	2 194,8
Бумага	177 руб.	3 уп.	449
Ручка	120 руб.	1 шт.	170
Заправка картриджа	850 руб.	4 раз	3 519
Итого:			7 743,8

3.3 Расчет затрат на разработку

Исходные данные, связанные с разработкой программного продукта, приведены в Таблице 3.5.

Таблица 3.5 — Исходные данные

Наименование	Обозначение	Значение
Оклад разработчиков (суммарный)	Ор	245 000 руб.
Время разработки	Трп	2 мес.
Машинное время разработки	Тмч	1,66 мес.
Коэффициент дополнительной заработной платы	Кд	0,15
Коэффициент страховых взносов	Кст	0,3
Количество единиц техники	Q	4 шт.
Себестоимость содержания техники	См/ч	22,3 руб./ч.
Коэффициент готовности техники	Кгт	0,97
Число рабочих дней в месяце	ЧРД	20 день
Продолжительность смены	Тст	8 ч.
Коэффициент сменности	Ксм	1
Коэффициент транспортных расходов	Кт	0,13
Коэффициент накладных расходов	Кнр	0,56

Расчет полных затрат на разработку проектного решения (программного продукта) осуществляется по формуле:

$$З_{рп} = З_{от} + З_{ст} + З_{эвм} + З_{сп} + З_{рм} + З_{нр}, \text{руб.}, \quad (3.4)$$

где $З_{от}$ — затраты на оплату труда разработчика (разработчиков), руб.;
(563 500)

$З_{ст}$ — страховые взносы по оплате труда во внебюджетные фонды, руб.;
(169 050)

$З_{эвм}$ — затраты, связанные с содержанием вычислительной техники, руб.; (23 015,38)

Зсп — затраты на технические средства и/или специальные программы, руб.; (97 631)

Зрм — затраты на расходные материалы, необходимые при разработке программного продукта, руб.; (8 750,49)

Знр — затраты по накладным расходам, приходящиеся на разработку программного продукта, руб. (315 560)

$$\begin{aligned} \text{Зрп} &= 563500 + 169050 + 23015,38 + 97631 + 8750,49 + 315560 \\ &= 1177506,87, \text{руб.}, \end{aligned} \quad (3.5)$$

3.3.1 Затраты на оплату труда разработчиков (Зот), руб.

Размер фонда оплаты труда разработчиков (Зот) рассчитывается по формуле:

$$\text{Зот} = \text{Ор} \times \text{Трп} (1 + \text{Кд}), \text{руб.}, \quad (3.6)$$

где Ор — месячный оклад разработчика проектного решения, руб./мес.;

Трп — время разработки проектного решения разработчиком, мес., включает в себя машинное время работы над проектом (Тмрп);

Кд — коэффициент дополнительной заработной платы разработчика.

$$\text{Зот} = 245000 \times 2 \times (1 + 0,15) = 563500, \text{руб.}, \quad (3.7)$$

3.3.2 Затраты по страховым взносам (Зст), руб.

Сумма страховых взносов определяется по формуле:

$$\text{Зст} = \text{Кст} \times \text{Зот}, \text{руб.}, \quad (3.8)$$

где Кст — коэффициент страховых взносов для расчета отчислений во внебюджетные фонды.

$$\text{Зст} = 0,3 \times 563500 = 169050, \text{руб.}, \quad (3.9)$$

3.3.3 Затраты по содержанию ЭВМ (Зэвм), руб

Затраты, связанные с эксплуатацией и содержанием ЭВМ, определяются по формуле:

$$\text{Зэвм} = \text{Тмрп} \times \text{Кгт} \times Q \times \text{См/ч, руб.}, \quad (3.10)$$

где Тмрп — машинное время на разработку проектного решения, ч.;

Кгт — коэффициент готовности техники;

Q — количество условных единиц используемой техники;

См/ч — стоимость машино-часа эксплуатации оборудования, руб. в ч.

$$\text{Зэвм} = 256 \times 0,97 \times 4 \times 22,3 = 23015,38, \text{руб.}, \quad (3.11)$$

Т.к. машинное время может измеряться в месяцах, а себестоимость машино-часа за один час, то машинное время необходимо перевести в часы.

Перевод рабочего времени в часы осуществляется по формуле:

$$\text{Тмрп} = \text{Тмч} \times \text{Чрд} \times \text{Тсм} \times \text{Ксм, ч.}, \quad (3.12)$$

где Тмч — рабочее время в месяцах;

Чрд — число рабочих дней в месяце;

Тсм — продолжительность рабочей смены;

Ксм — количество рабочих смен.

$$\text{Тмрп} = 1,66 \times 20 \times 8 \times 1 = 266, \text{ч.}, \quad (3.13)$$

3.3.4 Затраты на расходные материалы (Зрм), руб.

Затраты на расходные материалы, необходимые для разработки проектного решения, определяются по формуле:

$$\text{Зрм} = P (1 + \text{Кт}), \text{руб.}, \quad (3.14)$$

где P — затраты на расходные материалы, руб.;

Кт — коэффициент транспортных расходов.

$$З_{рм} = 7743,8 \times (1 + 0,13) = 8750,49, \text{руб.}, \quad (3.15)$$

3.3.5 Затраты по накладным расходам (Знр)

Затраты по накладным расходам определяются по формуле:

$$З_{нр} = К_{нр} \times З_{от}, \text{руб.}, \quad (3.16)$$

где $K_{нр}$ — коэффициент накладных расходов, принимается для расчета по данным предприятия;

$З_{от}$ — затраты по оплате труда, руб.

$$З_{нр} = 0,56 \times 563500 = 315560, \text{руб.}, \quad (3.17)$$

В Таблице 3.6 представлены результаты вычислений.

Таблица 3.6 — Исходные данные

Наименование	Обозначение	Значение, руб.
Оплата труда	Зот	563 500
Страховые взносы	Зст	169 050
Содержание ЭВМ	Зэвм	23 015,38
Затраты на технические средства	Зсп	97 631
Расходные материалы	Зрм	8 750,49
Накладные расходы	Знр	315 560
Итого затрат на разработку	Зрп	1 177 506,87

Произведя вычисления, было выявлено, что полные затраты на разработку составляют 1 177 506,87 рублей.

3.4 Расчет цены и прибыли

Исходные данные, связанные с разработкой программного продукта приведены в Таблице 3.7.

Таблица 3.7 — Исходные данные

Наименование	Обозначение	Значение
Коэффициент рентабельности	Кр	0,3
Коэффициент налога на добавочную стоимость	Кндс	0,22
Ставка налога на прибыль	Кнп	0,25
Стоимость затрат на разработку	Зрп	1 177 506,87руб.

Цена программного продукта, который разработан одной организацией по заказу другой и не предназначен для тиражирования, определяется по формуле:

$$\text{Цпп} = \text{Зрп} + \text{Ппл} + \text{НДС, руб.}, \quad (3.18)$$

где Ппл — планируемая прибыль рассчитывается по формуле:

$$\text{Ппл} = \text{Зрп} \times \text{Кр, руб.}, \quad (3.19)$$

$$\text{Ппл} = 1177506,87 \times 0,3 = 353252,06, \text{руб.}, \quad (3.20)$$

НДС — налог на добавленную стоимость определяется исходя из Кндс = 0,22 (ставка налога 22%) по формуле:

$$\text{НДС} = (\text{Зрп} + \text{Ппл}) \times \text{Кндс, руб.}, \quad (3.21)$$

$$\text{НДС} = (1177506,87 + 353252,06) \times 0,22 = 336766,96, \text{руб.}, \quad (3.22)$$

$$\text{Цпп} = 1177506,87 + 353252,06 + 336766,96 = 1851039,77, \text{руб.}, \quad (3.23)$$

Каждое предприятие с полученной прибыли перечисляет государству налог на прибыль. На сегодня ставка налога 25% ($K_{\text{нп}} = 0,25$) от полученной прибыли, и определяется по формуле:

$$\text{НП} = \text{Ппл} \times K_{\text{нп}}, \text{руб.}, \quad (3.24)$$

$$\text{НП} = 353252,06 \times 0,25 = 88313,02, \text{руб.}, \quad (3.25)$$

Чистая прибыль, остающаяся в распоряжении предприятия, рассчитывается по формуле:

$$\text{ЧП} = \text{Ппл} - \text{НП}, \text{руб.}, \quad (3.26)$$

$$\text{ЧП} = 353252,06 - 88313,02 = 264939,04, \text{руб.}, \quad (3.27)$$

Поступление в бюджет складывается из налога на прибыль и НДС, рассчитывается по формуле:

$$\text{ПБ} = \text{НП} + \text{НДС}, \text{руб.}, \quad (3.28)$$

$$\text{ПБ} = 88313,02 + 336766,96 = 425079,92, \text{руб.}, \quad (3.29)$$

Промежуточные результаты вычислений представлены в Таблице 3.8.

Таблица 3.8 — Результаты вычислений

Наименование	Обозначение	Значение, руб.
Плановая прибыль	Ппл	353 252,06
Налог на добавочную стоимость	НДС	336 766,96
Цена программного продукта	Цпп	1 851 039,77
Налог на прибыль	НП	88 313,02
Чистая прибыль	ЧП	264 939,04
Поступление в бюджет	ПБ	425 079,92

На основании произведенных расчетов можно сделать вывод, что на разработку продукта будет затрачено 1 177 506,87 рублей. Цена на разработку

продукта является приемлемой, так как с помощью него можно будет более эффективно выполнять поставленные задачи благодаря чему предприятие сможет взять больше заказов. Цена готового продукта будет равна 1 851 039,77 рубля, что в свою очередь позволит получить чистую прибыль в размере от 264 939,04 рублей.

ЗАКЛЮЧЕНИЕ

В результате выполнения дипломного проекта была разработана система управления проектами и задачами, предназначенная для автоматизации процессов планирования, распределения и контроля выполнения задач в рамках командной работы.

В ходе разработки были рассмотрены современные подходы к построению клиент-серверных приложений, выбраны технологии и архитектурные решения, обеспечивающие расширяемость, поддерживаемость и удобство дальнейшего сопровождения программного продукта. Серверная часть системы реализована на платформе ASP.NET Core с использованием архитектурного подхода Vertical Slice Architecture и принципов CQRS. Для организации взаимодействия между компонентами приложения использовался паттерн Mediator, реализованный с помощью библиотеки MediatR.

Разработанная система предоставляет возможности регистрации и авторизации пользователей, управления проектами, создания и редактирования задач, назначения исполнителей и отслеживания статусов выполнения задач. Для хранения данных использовалась PostgreSQL, обеспечивающая надежность хранения информации, поддержку реляционной модели данных, а также наличие расширенных типов данных, включая перечисления (enum).

В процессе реализации были разработаны REST и RPC интерфейсы взаимодействия клиента и сервера, DTO-модели передачи данных, механизмы валидации входных данных и обработки ошибок. Особое внимание уделялось разделению ответственности между компонентами системы, что позволило повысить читаемость и расширяемость кода.

Также было проведено тестирование программного продукта, в ходе которого подтверждена корректность работы основных бизнес-процессов

системы, стабильность выполнения пользовательских сценариев и корректная обработка исключительных ситуаций.

Результатом работы является функционирующий программный продукт, который может быть использован для интеграции с различными клиентскими решениями для организации командной работы и управления проектной деятельностью. Разработанная архитектура позволяет в дальнейшем расширять функциональные возможности системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Metanit. ASP.NET Core и Web API [Электронный ресурс] — URL: <https://metanit.com/sharp/aspnet6/> (Дата обращения: 18.04.2026)
- 2) Metanit. ASP.NET MVC. Введение в MVC [Электронный ресурс] — URL: <https://metanit.com/sharp/aspnetmvc/2.1.php> (Дата обращения: 22.04.2026)
- 3) Metanit. Платформа .NET и язык C# [Электронный ресурс] — URL: <https://metanit.com/sharp/tutorial/> (Дата обращения: 14.04.2026)
- 4) Metanit. PostgreSQL и язык SQL [Электронный ресурс] — URL: <https://metanit.com/sql/postgresql/> (Дата обращения: 21.05.2026)
- 5) PostgreSQL Global Development Group. PostgreSQL Documentation [Электронный ресурс] — URL: <https://www.postgresql.org/docs/> (Дата обращения: 23.05.2026)
- 6) Npgsql Documentation [Электронный ресурс] — URL: <https://www.npgsql.org/doc/> (Дата обращения: 24.05.2026)
- 7) Metanit. Entity Framework Core [Электронный ресурс] — URL: <https://metanit.com/sharp/efcore/> (Дата обращения: 28.04.2026)
- 8) Microsoft Learn. Entity Framework Core Documentation [Электронный ресурс] — URL: <https://learn.microsoft.com/ef/core/> (Дата обращения: 30.04.2026)
- 9) GitHub. MediatR Documentation [Электронный ресурс] — URL: <https://github.com/jbogard/MediatR> (Дата обращения: 08.05.2026)
- 10) Refactoring Guru. Mediator — паттерн проектирования [Электронный ресурс] — URL: <https://refactoring.guru/design-patterns/mediator> (Дата обращения: 03.05.2026)
- 11) Refactoring Guru. Mediator in C#: Example [Электронный ресурс] — URL: <https://refactoring.guru/design-patterns/mediator/csharp/example> (Дата обращения: 05.05.2026)
- 12) FluentValidation. Documentation [Электронный ресурс] — URL: <https://docs.fluentvalidation.net/> (Дата обращения: 10.05.2026)

13) Microsoft Learn. Authentication and authorization in ASP.NET Core [Электронный ресурс] — URL: <https://learn.microsoft.com/aspnet/core/security/authentication/> (Дата обращения: 13.05.2026)

14) Microsoft Learn. OpenAPI support in ASP.NET Core API apps [Электронный ресурс] — URL: <https://learn.microsoft.com/aspnet/core/fundamentals/openapi/> (Дата обращения: 15.05.2026)

15) Scalar. Scalar Documentation [Электронный ресурс] — URL: <https://scalar.com/docs> (Дата обращения: 17.05.2026)

16) xUnit.net. xUnit.net Documentation [Электронный ресурс] — URL: <https://xunit.net/> (Дата обращения: 19.05.2026)

17) Microsoft Learn. Vertical Slice Architecture in ASP.NET Core [Электронный ресурс] — URL: <https://learn.microsoft.com/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures> (Дата обращения: 25.04.2026)

18) MDN Web Docs. HTTP overview [Электронный ресурс] — URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview> (Дата обращения: 26.05.2026)

19) AWS. What is RESTful API? [Электронный ресурс] — URL: <https://aws.amazon.com/what-is/restful-api/> (Дата обращения: 27.05.2026)

20) Microsoft Learn. gRPC services with ASP.NET Core [Электронный ресурс] — URL: <https://learn.microsoft.com/aspnet/core/grpc/> (Дата обращения: 28.05.2026)

Приложение А

Код моделей

Листинг А.1 — User.cs

```
using ProjectManagement.Domain.Enums;
using ProjectManagement.Domain.Interfaces;
using ProjectManagement.Domain.ValueObjects;

namespace ProjectManagement.Domain.Models;

public class User : IAuditable
{
    public const int NameMaxLength = 50;

    private string _name = null!;
    private Email _email = null!;
    private string _passwordHash = null!;

    public User(string name, Email email, string passwordHash, UserRole role)
    {
        Name = name;
        Email = email;
        PasswordHash = passwordHash;
        Role = role;
    }

    private User()
    {
    }

    public Guid Id { get; private init; } = Guid.CreateVersion7();

    public string Name
    {
        get => _name;
        private set
        {
            if (string.IsNullOrEmpty(value)) throw new
ArgumentNullException(nameof(Name));
            if (value.Length > NameMaxLength) throw new
ArgumentOutOfRangeException(nameof(Name));
            _name = value;
        }
    }

    public Email Email
    {
        get => _email;
        private set => _email = value ?? throw new
ArgumentNullException(nameof(Email));
    }

    public string PasswordHash
    {
        get => _passwordHash;
        private set
```

```

        {
            if (string.IsNullOrEmpty(value)) throw new
ArgumentNullException(nameof(PasswordHash));
            _passwordHash = value;
        }
    }

    public UserRole Role { get; private set; }

    public void Rename(string newName) => Name = newName;
    public void ChangeEmail(Email newEmail) => Email = newEmail;
    public void ChangePassword(string newPasswordHash) => PasswordHash =
newPasswordHash;
    public void ChangeRole(UserRole newRole) => Role = newRole;
}

```

Листинг A.2 — WorkTask.cs

```

using ProjectManagement.Domain.Enums;
using ProjectManagement.Domain.Exceptions;
using ProjectManagement.Domain.Interfaces;

namespace ProjectManagement.Domain.Models;

public class WorkTask : IAuditable
{
    public const int MaxTitleLength = 400;
    public const int MaxDescriptionLength = 2000;

    private string _title = null!;
    private string? _description;
    private DateTime? _deadline;

    private WorkTask(Guid projectId, string title, string? description)
    {
        ProjectId = projectId;
        Title = title;
        Description = description;
    }

    private WorkTask()
    {
    }

    public Guid Id { get; private init; } = Guid.CreateVersion7();

    public string Title
    {
        get => _title;
        private set
        {
            if (string.IsNullOrEmpty(value)) throw new
ArgumentNullException(nameof(Title));
            if (value.Length > MaxTitleLength) throw new
ArgumentOutOfRangeException(nameof(Title));
            _title = value;
        }
    }
}

```

```

public string? Description
{
    get => _description;
    private set
    {
        if (value is not null && value.Length > MaxDescriptionLength)
            throw new ArgumentOutOfRangeException(
                nameof(Description),
                $"Max length is {MaxDescriptionLength} characters."
            );
        _description = value;
    }
}

public DateTime? Deadline
{
    get => _deadline;
    private set
    {
        if (value.HasValue && value.Value <= DateTime.UtcNow)
            throw new ArgumentOutOfRangeException(
                nameof(Deadline),
                "Deadline cannot be in the past."
            );
        _deadline = value;
    }
}

public WorkTaskStatus Status { get; private set; } = WorkTaskStatus.Active;

public Guid ProjectId { get; private set; }
public Guid? ExecutorId { get; private set; }

public static WorkTask Create(Guid projectId, string title, string? description,
DateTime? deadline)
{
    var workTask = new WorkTask(projectId, title, description);
    workTask.SetDeadline(deadline);
    return workTask;
}

internal void Complete()
{
    if (Status == WorkTaskStatus.Completed) return;
    if (ExecutorId is null) throw new DomainRuleException("The task cannot be
completed without an executor.");
    Status = WorkTaskStatus.Completed;
}

internal void Rename(string newTitle)
{
    EnsureEditable();
    Title = newTitle;
}

internal void ChangeDescription(string? newDescription)
{
    EnsureEditable();
    Description = newDescription;
}

```

```

internal void SetDeadline(DateTime? deadline)
{
    EnsureEditable();
    Deadline = deadline;
}

internal void AssignExecutor(Guid userId)
{
    EnsureEditable();
    ExecutorId = userId;
}

internal void UnassignExecutor()
{
    ExecutorId = null;
}

private void EnsureEditable()
{
    if (Status == WorkTaskStatus.Completed) throw new DomainRuleException("Cannot
change completed task.");
}
}

```

Листинг А.3 — Project.cs

```

using ProjectManagement.Domain.Enums;
using ProjectManagement.Domain.Exceptions;
using ProjectManagement.Domain.Interfaces;

namespace ProjectManagement.Domain.Models;

public class Project : IAuditable
{
    public const int MaxTitleLength = 400;
    public const int MaxDescriptionLength = 2000;

    private string _title = null!;
    private string? _description;

    private readonly List<WorkTask> _workTasks = [];
    private readonly List<ProjectExecutor> _executors = [];

    public Project(string title, string? description, ProjectPriority priority)
    {
        Title = title;
        Description = description;
        Priority = priority;
    }

    private Project()
    {
    }

    public Guid Id { get; private init; } = Guid.CreateVersion7();

    public Guid? ManagerId { get; private set; }

    public string Title
    {

```

```

        get => _title;
        private set
        {
            if (string.IsNullOrEmpty(value)) throw new
ArgumentNullException(nameof(Title));
            if (value.Length > MaxTitleLength) throw new
ArgumentOutOfRangeException(nameof(Title));
            _title = value;
        }
    }

    public string? Description
    {
        get => _description;
        private set
        {
            if (value is not null && value.Length > MaxDescriptionLength)
                throw new ArgumentOutOfRangeException(
                    nameof(Description),
                    $"Max length is {MaxDescriptionLength} characters."
                );

            _description = value;
        }
    }

    public ProjectPriority Priority { get; private set; } = ProjectPriority.Medium;
    public ProjectStatus Status { get; private set; } = ProjectStatus.Active;

    public IReadOnlyCollection<WorkTask> WorkTasks => _workTasks;
    public IReadOnlyCollection<ProjectExecutor> Executors => _executors;

    public void ChangeTitle(string newTitle)
    {
        EnsureEditable();
        Title = newTitle;
    }

    public void ChangeDescription(string? newDescription)
    {
        EnsureEditable();
        Description = newDescription;
    }

    public void AssignManager(Guid userId)
    {
        EnsureEditable();
        ManagerId = userId;
    }

    public void UnassignManager()
    {
        ManagerId = null;
    }

    public void AddExecutor(Guid userId)
    {
        EnsureEditable();

        if (ManagerId == userId) throw new DomainRuleException("Manager cannot be
project executor.");
    }

```

```

        if (!_executors.All(x => x.UserId != userId))
        {
            _executors.Add(new ProjectExecutor(Id, userId));
        }
    }

    public void RemoveExecutor(Guid userId)
    {
        if (_workTasks.Any(x => x.ExecutorId == userId && x.Status !=
WorkTaskStatus.Completed))
        {
            throw new DomainRuleException("This executor has active tasks.");
        }

        foreach (var task in _workTasks.Where(x => x.ExecutorId == userId))
        {
            task.UnassignExecutor();
        }

        _executors.RemoveAll(x => x.UserId == userId);
    }

    public void Complete()
    {
        EnsureEditable();

        if (_workTasks.Any(x => x.Status != WorkTaskStatus.Completed))
        {
            throw new DomainRuleException("The project contains uncompleted tasks.");
        }

        Status = ProjectStatus.Completed;
    }

    public void CreateWorkTask(string title, string? description, DateTime? deadline)
    {
        EnsureEditable();

        var workTask = WorkTask.Create(Id, title, description, deadline);

        _workTasks.Add(workTask);
    }

    public void RemoveWorkTask(Guid workTaskId)
    {
        EnsureEditable();

        var workTask = GetWorkTask(workTaskId);
        _workTasks.Remove(workTask);
    }

    public void AssignWorkTaskExecutor(Guid workTaskId, Guid userId)
    {
        EnsureEditable();

        var workTask = GetWorkTask(workTaskId);

        if (!_executors.All(x => x.UserId != userId))
            throw new DomainRuleException("This user is not a project executor.");

        workTask.AssignExecutor(userId);
    }

```

```

public void UnassignWorkTaskExecutor(Guid workTaskId)
{
    var workTask = GetWorkTask(workTaskId);
    workTask.UnassignExecutor();
}

public void RenameWorkTask(Guid workTaskId, string newTitle)
{
    EnsureEditable();

    var workTask = GetWorkTask(workTaskId);
    workTask.Rename(newTitle);
}

public void ChangeWorkTaskDescription(Guid workTaskId, string newDescription)
{
    EnsureEditable();

    var workTask = GetWorkTask(workTaskId);
    workTask.ChangeDescription(newDescription);
}

public void ChangeWorkTaskDeadline(Guid workTaskId, DateTime? deadline)
{
    EnsureEditable();

    var workTask = GetWorkTask(workTaskId);
    workTask.SetDeadline(deadline);
}

public void CompleteWorkTask(Guid workTaskId)
{
    EnsureEditable();

    var workTask = GetWorkTask(workTaskId);
    workTask.Complete();
}

private WorkTask GetWorkTask(Guid workTaskId)
{
    return _workTasks.FirstOrDefault(x => x.Id == workTaskId)
        ?? throw new DomainRuleException("This task is not in the project.");
}

private void EnsureEditable()
{
    if (Status == ProjectStatus.Completed) throw new DomainRuleException("Cannot
change completed project.");
}
}

```

Листинг А.4 — ProjectExecutor.cs

```

namespace ProjectManagement.Domain.Models;

public class ProjectExecutor
{
    public Guid ProjectId { get; private init; }
    public Guid UserId { get; private init; }
}

```



```

        internal ProjectExecutor(Guid projectId, Guid userId)
        {
            ProjectId = projectId;
            UserId = userId;
        }

        private ProjectExecutor()
        {
        }
    }

```

Листинг A.5 — IAuditable.cs

```

namespace ProjectManagement.Domain.Interfaces;

public interface IAuditable
{
}

```

Листинг A.6 — UserRole.cs

```

namespace ProjectManagement.Domain.Enums;

public enum UserRole
{
    Employee = 0,
    ProjectManager = 1,
    Admin = 2,
    Supervisor = 3,
}

```

Листинг A.7 — WorkTaskStatus.cs

```

namespace ProjectManagement.Domain.Enums;

public enum WorkTaskStatus
{
    Active = 0,
    Completed = 1,
}

```

Листинг A.8 — ProjectPriority.cs

```

namespace ProjectManagement.Domain.Enums;

public enum ProjectPriority
{
    Medium = 0,
    Low = 1,
    High = 2,
}

```

Листинг A.9 — ProjectStatus.cs

```

namespace ProjectManagement.Domain.Enums;

public enum ProjectStatus
{
    Active = 0,
    Completed = 1,
}

```

Листинг A.10 — Email.cs

```

using System.Text.RegularExpressions;

```

```

using ProjectManagement.Domain.Exceptions;

namespace ProjectManagement.Domain.ValueObjects;

public sealed record Email
{
    public const int MaxLenght = 50;

    public const string Pattern =
        @"^(?!.*(\.\.|\_|\+|\+|--))[a-z0-9][a-z0-9._+-]+[a-z0-9]@[a-z][a-z0-9.-]+[a-
z0-9]\.[a-z]{2,}$";

    private static readonly Regex Regex = new(
        Pattern,
        RegexOptions.Compiled
    );

    private Email(string value) => Value = value;

    public string Value { get; }

    public static Email Create(string value)
    {
        if (string.IsNullOrEmpty(value))
            throw new EmailException("Email is empty.");

        if (value.Length > MaxLenght)
            throw new EmailException("Email is too long.");

        value = value.Trim().ToLower();

        if (!Regex.IsMatch(value))
            throw new EmailException("Invalid email.");

        return new Email(value);
    }

    public static implicit operator string(Email email) => email.Value;
    public override string ToString() => Value;
}

```

Приложение Б

Код Entity Framework Core конфигураций

Листинг Б.1 — UserConfiguration.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using ProjectManagement.Domain.Models;
using ProjectManagement.Domain.ValueObjects;
using ProjectManagement.Infrastructure.Persistence.Extensions;

namespace ProjectManagement.Infrastructure.Persistence.Configurations;

public class UserConfiguration : IEntityTypeConfiguration<User>
{
    public void Configure(EntityTypeBuilder<User> builder)
    {
        builder.ToTable("users");

        builder.HasKey(x => x.Id);

        builder.Property(x => x.Name)
            .HasMaxLength(User.NameMaxLength)
            .IsRequired();

        builder.Property(x => x.Email)
            .HasConversion(
                e => e.Value,
                s => Email.Create(s)
            )
            .HasMaxLength(Email.MaxLength)
            .IsRequired();

        builder.Property(x => x.PasswordHash)
            .IsRequired();

        builder.Property(x => x.Role)
            .IsRequired();

        builder.HasIndex(x => x.Email)
            .IsUnique();

        builder.AddAudit();
    }
}
```

Листинг Б.2 — WorkTaskConfiguration.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using ProjectManagement.Domain.Models;
using ProjectManagement.Infrastructure.Persistence.Extensions;

namespace ProjectManagement.Infrastructure.Persistence.Configurations;

public class WorkTaskConfiguration : IEntityTypeConfiguration<WorkTask>
{
    public void Configure(EntityTypeBuilder<WorkTask> builder)
    {

```

```

        builder.ToTable("work_tasks");

        builder.HasKey(x => x.Id);

        builder.Property(x => x.Title)
            .HasMaxLength(WorkTask.MaxTitleLength)
            .IsRequired();

        builder.Property(x => x.Description)
            .HasMaxLength(WorkTask.MaxDescriptionLength)
            .IsRequired(false);

        builder.Property(x => x.Deadline)
            .HasColumnType("timestampz")
            .HasPrecision(0)
            .IsRequired(false);

        builder.Property(x => x.Status)
            .IsRequired();

        builder.HasOne<Project>()
            .WithMany(x => x.WorkTasks)
            .HasForeignKey(x => x.ProjectId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();

        builder.HasOne<User>()
            .WithMany()
            .HasForeignKey(x => x.ExecutorId)
            .OnDelete(DeleteBehavior.SetNull)
            .IsRequired(false);

        builder.HasIndex(x => new { x.ProjectId, x.Title })
            .IsUnique();

        builder.HasIndex(x => x.ExecutorId);

        builder.AddAudit();
    }
}

```

Листинг Б.3 — ProjectConfiguration.cs

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using ProjectManagement.Domain.Models;
using ProjectManagement.Infrastructure.Persistence.Extensions;

namespace ProjectManagement.Infrastructure.Persistence.Configurations;

public class ProjectConfiguration : IEntityTypeConfiguration<Project>
{
    public void Configure(EntityTypeBuilder<Project> builder)
    {
        builder.ToTable("projects");

        builder.HasKey(x => x.Id);

        builder.Property(x => x.ManagerId)
            .IsRequired(false);

        builder.Property(x => x.Title)

```

```

        .HasMaxLength(Project.MaxTitleLength)
        .IsRequired();

builder.Property(x => x.Description)
    .HasMaxLength(Project.MaxDescriptionLength)
    .IsRequired(false);

builder.Property(x => x.Priority)
    .IsRequired();

builder.Property(x => x.Status)
    .IsRequired();

builder.Navigation(x => x.WorkTasks)
    .HasField("_workTasks")
    .UsePropertyAccessMode(PropertyAccessMode.Field);

builder.Navigation(x => x.Executors)
    .HasField("_executors")
    .UsePropertyAccessMode(PropertyAccessMode.Field);

builder.HasIndex(x => new { x.ManagerId, x.Title })
    .IsUnique();

builder.AddAudit();
    }
}

```

Листинг Б.4 — ProjectExecutorConfiguration.cs

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using ProjectManagement.Domain.Models;

namespace ProjectManagement.Infrastructure.Persistence.Configurations;

public class ProjectExecutorConfiguration : IEntityTypeConfiguration<ProjectExecutor>
{
    public void Configure(EntityTypeBuilder<ProjectExecutor> builder)
    {
        builder.ToTable("project_executors");

        builder.HasKey(x => new { x.ProjectId, x.UserId });

        builder.HasOne<Project>()
            .WithMany(x => x.Executors)
            .HasForeignKey(x => x.ProjectId);

        builder.HasOne<User>()
            .WithMany()
            .HasForeignKey(x => x.UserId);
    }
}

```

Листинг Б.5 — AuditConfigurationExtensions.cs

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace ProjectManagement.Infrastructure.Persistence.Extensions;

public static class AuditConfigurationExtensions
{

```

```

public static void AddAudit<TEntity>(this EntityTypeBuilder<TEntity> builder)
    where TEntity : class
{
    builder.Property<DateTime>("CreatedAt")
        .HasColumnType("timestampz")
        .HasPrecision(0)
        .IsRequired();

    builder.Property<DateTime>("UpdatedAt")
        .HasColumnType("timestampz")
        .HasPrecision(0)
        .IsRequired();
}
}

```

Листинг Б.6 — ProjectManagementDbContext.cs

```

using Microsoft.EntityFrameworkCore;
using ProjectManagement.Domain.Enums;
using ProjectManagement.Domain.Models;

namespace ProjectManagement.Infrastructure.Persistence;

public class ProjectManagementDbContext(DbContextOptions<ProjectManagementDbContext>
options)
    : DbContext(options)
{
    public DbSet<User> Users => Set<User>();
    public DbSet<Project> Projects => Set<Project>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.HasPostgresEnum<UserRole>();
        modelBuilder.HasPostgresEnum<WorkTaskStatus>();
        modelBuilder.HasPostgresEnum<ProjectPriority>();
        modelBuilder.HasPostgresEnum<ProjectStatus>();

        modelBuilder.ApplyConfigurationsFromAssembly(typeof(ProjectManagementDbContext).Assembly);
    }
}

```

Листинг Б.7 — AuditInterceptor.cs

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Diagnostics;
using ProjectManagement.Domain.Interfaces;

namespace ProjectManagement.Infrastructure.Persistence.Interceptors;

public class AuditInterceptor : SaveChangesInterceptor
{
    public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData,
        InterceptionResult<int> result,
        CancellationToken cancellationToken = default)
    {
        var context = eventData.Context;

        if (context is null)

```

```

        return base.SavingChangesAsync(eventData, result, cancellationToken);

var now = DateTime.UtcNow;

var entries = context.ChangeTracker.Entries<IAuditable>()
    .Where(e => e.State is EntityState.Added or EntityState.Modified);

foreach (var entry in entries)
{
    if (entry.State == EntityState.Added)
    {
        entry.Property("CreatedAt").CurrentValue = now;
    }

    entry.Property("UpdatedAt").CurrentValue = now;
}

return base.SavingChangesAsync(eventData, result, cancellationToken);
}
}

```

Листинг Б.8 — DependencyInjection.cs

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using ProjectManagement.Domain.Enums;
using ProjectManagement.Infrastructure.Persistence.Interceptors;

namespace ProjectManagement.Infrastructure.Persistence.Extensions;

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services,
        IConfiguration configuration)
    {
        var connectionString =
            configuration.GetConnectionString("Default")
            ?? throw new InvalidOperationException("Connection string not found.");

        services.AddScoped<AuditInterceptor>();

        services.AddDbContext<ProjectManagementDbContext>((sp, options) =>
        {
            options
                .UseNpgsql(
                    connectionString,
                    o =>
                    {
                        o.MapEnum<UserRole>();
                        o.MapEnum<WorkTaskStatus>();
                        o.MapEnum<ProjectPriority>();
                        o.MapEnum<ProjectStatus>();
                    })
                .UseSnakeCaseNamingConvention()
                .AddInterceptors(sp.GetRequiredService<AuditInterceptor>());
        });

        return services;
    }
}

```

Приложение В

Код среза функции регистрации

Листинг В.1 — RegisterResponse.cs

```
namespace ProjectManagement.Api.Features.Auth.Register;

public record RegisterResponse(Guid UserId);
```

Листинг В.2 — RegisterCommand.cs

```
using MediatR;
using ProjectManagement.Domain.Enums;

namespace ProjectManagement.Api.Features.Auth.Register;

public sealed record RegisterCommand(
    string Name,
    string Email,
    string Password,
    UserRole Role
) : IRequest<RegisterResponse>;
```

Листинг В.3 — Validator.cs

```
using FluentValidation;
using ProjectManagement.Domain.Models;
using ProjectManagement.Domain.ValueObjects;

namespace ProjectManagement.Api.Features.Auth.Register;

public sealed class Validator : AbstractValidator<RegisterCommand>
{
    public Validator()
    {
        RuleFor(x => x.Name)
            .NotEmpty()
            .MaximumLength(User.NameMaxLength);

        RuleFor(x => x.Email)
            .NotEmpty()
            .MaximumLength(Email.MaxLength)
            .Matches(Email.Pattern);

        RuleFor(x => x.Password)
            .NotEmpty()
            .MinimumLength(8);

        RuleFor(x => x.Role)
            .IsInEnum();
    }
}
```

Листинг В.4 — Handler.cs

```
using MediatR;
using Microsoft.EntityFrameworkCore;
using ProjectManagement.Domain.Models;
using ProjectManagement.Domain.ValueObjects;
using ProjectManagement.Infrastructure.Persistence;
```



```

namespace ProjectManagement.Api.Features.Auth.Register;

public sealed class Handler(ProjectManagementDbContext db)
    : IRequestHandler<RegisterCommand, RegisterResponse>
{
    public async Task<RegisterResponse> Handle(
        RegisterCommand request,
        Cancellation_token ct)
    {
        var exists = await db.Users
            .AnyAsync(x => x.Email == request.Email, ct);

        if (exists)
            throw new("User already exists.");

        var passwordHash =
            BCrypt.Net.BCrypt.HashPassword(request.Password);

        var user = new User(
            request.Name,
            Email.Create(request.Email),
            passwordHash,
            request.Role);

        db.Users.Add(user);

        await db.SaveChangesAsync(ct);

        return new RegisterResponse(user.Id);
    }
}

```

Листинг В.5 —Endpoint.cs

```

using MediatR;
using Microsoft.AspNetCore.Http.HttpResults;
using Microsoft.AspNetCore.Mvc;
using ProjectManagement.Domain.Enums;

namespace ProjectManagement.Api.Features.Auth.Register;

public static class Endpoint
{
    public static RouteGroupBuilder MapRegister(this RouteGroupBuilder group)
    {
        group.MapPost("register", Handle)
            .RequireAuthorization(nameof(UserRole.Admin));

        return group;
    }

    private static async Task<Ok<RegisterResponse>> Handle(
        [FromBody] RegisterCommand command,
        [FromServices] ISender sender,
        Cancellation_token ct)
    {
        var response = await sender.Send(command, ct);
        return TypedResults.Ok(response);
    }
}

```

Приложение Г

Код реализации аутентификации и авторизации

Листинг Г.1 — JwtProvider.cs

```
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using ProjectManagement.Api.Shared;
using ProjectManagement.Domain.Models;

namespace ProjectManagement.Api.Services;

public class JwtProvider(IOptions<JwtSettings> options)
{
    public string Generate(User user)
    {
        var claims = new[]
        {
            new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
            new Claim(ClaimTypes.Role, user.Role.ToString()),
        };

        var signingKey = new SymmetricSecurityKey(
            Convert.FromBase64String(options.Value.Key));

        var credentials = new SigningCredentials(
            signingKey,
            SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: options.Value.Issuer,
            claims: claims,
            expires: DateTime.UtcNow.Add(options.Value.Expires),
            signingCredentials: credentials);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

Листинг Г.2 — AuthenticationExtensions.cs

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using ProjectManagement.Api.Services;
using ProjectManagement.Api.Shared;
using ProjectManagement.Domain.Enums;

namespace ProjectManagement.Api.Extensions;

public static class AuthenticationExtensions
{
    public static IServiceCollection AddAuthenticationConfigured(
        this IServiceCollection services,
        IConfiguration configuration)
    {
        services.Configure<JwtSettings>(configuration.GetSection(nameof(JwtSettings)));
    }
}
```

```

services.AddSingleton<JwtProvider>();

var jwtSettings =
configuration.GetSection(nameof(JwtSettings)).Get<JwtSettings>();
if (jwtSettings == null) throw new
ArgumentNullException(nameof(jwtSettings));

services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
.AddJwtBearer(options =>
{
    options.TokenValidationParameters =
        new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = false,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,

            ValidIssuer = jwtSettings.Issuer,

            IssuerSigningKey = new SymmetricSecurityKey(
                Convert.FromBase64String(jwtSettings.Key)),
        };
});

services.AddAuthorization(options =>
{
    options.AddPolicy(nameof(UserRole.Employee), policy =>
policy.RequireRole(nameof(UserRole.Employee)));
    options.AddPolicy(nameof(UserRole.ProjectManager),
        policy => policy.RequireRole(nameof(UserRole.ProjectManager)));
    options.AddPolicy(nameof(UserRole.Admin), policy =>
policy.RequireRole(nameof(UserRole.Admin)));
    options.AddPolicy(nameof(UserRole.Supervisor), policy =>
policy.RequireRole(nameof(UserRole.Supervisor)));
});

return services;
}
}

```

Приложение Д

Код тестов

Листинг Д.1 — EmailTests.cs

```
using ProjectManagement.Domain.Exceptions;
using ProjectManagement.Domain.ValueObjects;

namespace ProjectManagement.Domain.Tests.ValueObjects;

public class EmailTests
{
    [Fact]
    public void Create_Should_Normalize_Email()
    {
        var email = Email.Create(" TEST1@MAIL.COM ");

        Assert.Equal("test1@mail.com", email.Value);
    }

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData(null)]
    public void Create_Should_Throw_When_Email_Is_Empty(string? value)
    {
        Assert.Throws<EmailException>(() => Email.Create(value!));
    }

    [Theory]
    [InlineData("invalid")]
    [InlineData("@mail.com")]
    [InlineData("test@")]
    [InlineData("test@mail.c")]
    [InlineData("test..test@mail.com")]
    [InlineData(".test@mail.com")]
    [InlineData("te$t@mail.com")]
    public void Create_Should_Throw_When_Email_Has_Invalid_Format(string value)
    {
        Assert.Throws<EmailException>(() => Email.Create(value));
    }

    [Fact]
    public void Create_Should_Throw_When_Email_Is_Too_Long()
    {
        var value = $"{new string('a', Email.MaxLength)}@mail.com";

        Assert.Throws<EmailException>(() => Email.Create(value));
    }
}
```

Листинг Д.2 — UserTests.cs

```
using ProjectManagement.Domain.Enums;
using ProjectManagement.Domain.Models;
using ProjectManagement.Domain.ValueObjects;

namespace ProjectManagement.Domain.Tests.Users;
```

```

public class UserTests
{
    [Fact]
    public void Constructor_Should_Create_User_With_Valid_Data()
    {
        var user = CreateUser();

        Assert.NotEqual(Guid.Empty, user.Id);
        Assert.Equal("test_name", user.Name);
        Assert.Equal("test@mail.com", user.Email.Value);
        Assert.Equal("password_hash", user.PasswordHash);
        Assert.Equal(UserRole.Employee, user.Role);
    }

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData(null)]
    public void Rename_Should_Throw_When_Name_Is_Empty(string? name)
    {
        var user = CreateUser();

        Assert.Throws<ArgumentNullException>(() => user.Rename(name!));
    }

    [Fact]
    public void Rename_Should_Throw_When_Name_Is_Too_Long()
    {
        var user = CreateUser();

        var longName = new string('a', User.NameMaxLength + 1);

        Assert.Throws<ArgumentOutOfRangeException>(() => user.Rename(longName));
    }

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData(null)]
    public void ChangePassword_Should_Throw_When_PasswordHash_Is_Empty(string?
password)
    {
        var user = CreateUser();

        Assert.Throws<ArgumentNullException>(() => user.ChangePassword(password!));
    }

    [Fact]
    public void ChangeEmail_Should_Throw_When_Email_Is_Null()
    {
        var user = CreateUser();

        Assert.Throws<ArgumentNullException>(() => user.ChangeEmail(null!));
    }

    private static User CreateUser()
    {
        return new User(
            "test_name",
            Email.Create("test@mail.com"),
            "password_hash",
            UserRole.Employee
        );
    }
}

```

```

    });
}
}

```

Листинг Д.3 — WorkTaskTests.cs

```

using ProjectManagement.Domain.Enums;
using ProjectManagement.Domain.Exceptions;
using ProjectManagement.Domain.Models;

namespace ProjectManagement.Domain.Tests.WorkTasks;

public class WorkTaskTests
{
    [Fact]
    public void Create_Should_Set_Default_State()
    {
        var task = CreateTask();

        Assert.Equal(WorkTaskStatus.Active, task.Status);
        Assert.Null(task.ExecutorId);
    }

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    public void Create_Should_Throw_When_Title_Is_Empty(string title)
    {
        Assert.Throws<ArgumentNullException>(() =>
            WorkTask.Create(
                Guid.CreateVersion7(),
                title,
                "desc",
                DateTime.UtcNow.AddDays(1))
        );
    }

    [Fact]
    public void Create_Should_Throw_When_Title_Too_Long()
    {
        var title = new string('a', WorkTask.MaxTitleLength + 1);

        Assert.Throws<ArgumentOutOfRangeException>(() =>
            WorkTask.Create(
                Guid.CreateVersion7(),
                title,
                "desc",
                DateTime.UtcNow.AddDays(1))
        );
    }

    [Fact]
    public void SetDeadline_Should_Throw_When_In_Past()
    {
        var task = CreateTask();

        Assert.Throws<ArgumentOutOfRangeException>(() =>
            task.SetDeadline(DateTime.UtcNow.AddDays(-1))
        );
    }

    [Fact]

```

```

public void Complete_Should_Throw_When_No_Executor()
{
    var task = CreateTask();

    Assert.Throws<DomainRuleException>(() => task.Complete());
}

[Fact]
public void Complete_Should_Be_Idempotent()
{
    var task = CreateTask();

    var id = Guid.CreateVersion7();

    task.AssignExecutor(id);

    task.Complete();
    task.Complete();

    Assert.Equal(WorkTaskStatus.Completed, task.Status);
}

[Fact]
public void Rename_Should_Throw_When_Completed()
{
    var task = CreateCompletedTask();

    Assert.Throws<DomainRuleException>(() => task.Rename("new"));
}

[Fact]
public void UnassignExecutor_Should_Work_Even_When_Completed()
{
    var task = CreateCompletedTask();

    task.UnassignExecutor();

    Assert.Null(task.ExecutorId);
}

private static WorkTask CreateTask()
{
    return WorkTask.Create(
        Guid.CreateVersion7(),
        "task",
        "desc",
        DateTime.UtcNow.AddDays(1)
    );
}

private static WorkTask CreateCompletedTask()
{
    var t = CreateTask();
    t.AssignExecutor(Guid.CreateVersion7());
    t.Complete();
    return t;
}
}

```

Листинг Д.4 — ProjectTests.cs

```
using ProjectManagement.Domain.Enums;
using ProjectManagement.Domain.Exceptions;
using ProjectManagement.Domain.Models;

namespace ProjectManagement.Domain.Tests.Projects;

public class ProjectTests
{
    [Fact]
    public void Constructor_Should_Set_Default_State()
    {
        var project = ProjectTestData.CreateProject();

        Assert.Equal(ProjectStatus.Active, project.Status);
        Assert.Equal(ProjectPriority.Medium, project.Priority);
        Assert.Empty(project.WorkTasks);
        Assert.Empty(project.Executors);
    }

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    public void Constructor_Should_Throw_When_Title_Is_Empty(string title)
    {
        Assert.Throws<ArgumentNullException>(() =>
            new Project(title, "desc", ProjectPriority.Medium)
        );
    }

    [Fact]
    public void Constructor_Should_Throw_When_Title_Too_Long()
    {
        var title = new string('a', Project.MaxTitleLength + 1);

        Assert.Throws<ArgumentOutOfRangeException>(() =>
            new Project(title, "desc", ProjectPriority.Medium)
        );
    }

    [Fact]
    public void Constructor_Should_Throw_When_Description_Too_Long()
    {
        var description = new string('a', Project.MaxDescriptionLength + 1);

        Assert.Throws<ArgumentOutOfRangeException>(() =>
            new Project("title", description, ProjectPriority.Medium)
        );
    }

    [Fact]
    public void Complete_Should_Throw_When_Has_Incomplete_Tasks()
    {
        var project = ProjectTestData.CreateProject();

        project.CreateWorkTask("t", null, DateTime.UtcNow.AddDays(1));

        Assert.Throws<DomainRuleException>(() => project.Complete());
    }

    [Fact]
```



```

public void Complete_Should_Work_When_All_Tasks_Completed()
{
    var project = ProjectTestData.CreateProject();

    project.AddExecutor(Guid.CreateVersion7());
    var user = project.Executors.First().UserId;

    project.CreateWorkTask("t", null, DateTime.UtcNow.AddDays(1));
    var task = project.WorkTasks.First();

    project.AssignWorkTaskExecutor(task.Id, user);
    project.CompleteWorkTask(task.Id);

    project.Complete();

    Assert.Equal(ProjectStatus.Completed, project.Status);
}

[Fact]
public void ChangeTitle_Should_Throw_When_Completed()
{
    var project = ProjectTestData.CreateCompletedProject();

    Assert.Throws<DomainRuleException>(() => project.ChangeTitle("new"));
}
}

```

Листинг Д.5 — ProjectWorkTasksTests.cs

```

using ProjectManagement.Domain.Exceptions;

namespace ProjectManagement.Domain.Tests.Projects;

public class ProjectWorkTasksTests
{
    [Fact]
    public void CreateWorkTask_Should_Add_Task()
    {
        var project = ProjectTestData.CreateProject();

        project.CreateWorkTask("Task", null, DateTime.UtcNow.AddDays(1));

        Assert.Single(project.WorkTasks);
    }

    [Fact]
    public void CreateWorkTask_Should_Throw_When_Project_Completed()
    {
        var project = ProjectTestData.CreateCompletedProject();

        Assert.Throws<DomainRuleException>(() =>
            project.CreateWorkTask("Task", null, DateTime.UtcNow.AddDays(1))
        );
    }

    [Fact]
    public void AssignExecutor_Should_Throw_When_User_Not_In_Project()
    {
        var project = ProjectTestData.CreateProject();

        project.CreateWorkTask("Task", null, DateTime.UtcNow.AddDays(1));
    }
}

```

```

        Assert.Throws<DomainRuleException>(() =>
            project.AssignWorkTaskExecutor(project.WorkTasks.First().Id,
            Guid.CreateVersion7())
        );
    }

    [Fact]
    public void AssignExecutor_Should_Work_When_User_In_Project()
    {
        var project = ProjectTestData.CreateProject();

        var user = Guid.CreateVersion7();
        project.AddExecutor(user);

        project.CreateWorkTask("Task", null, DateTime.UtcNow.AddDays(1));

        var task = project.WorkTasks.First();

        project.AssignWorkTaskExecutor(task.Id, user);

        Assert.Equal(user, task.ExecutorId);
    }

    [Fact]
    public void RemoveWorkTask_Should_Delete_Task()
    {
        var project = ProjectTestData.CreateProject();

        project.CreateWorkTask("Task", null, DateTime.UtcNow.AddDays(1));

        var task = project.WorkTasks.First();

        project.RemoveWorkTask(task.Id);

        Assert.Empty(project.WorkTasks);
    }
}

```

Листинг Д.6 — ProjectExecutorsTests.cs

```

using ProjectManagement.Domain.Exceptions;

namespace ProjectManagement.Domain.Tests.Projects;

public class ProjectExecutorsTests
{
    [Fact]
    public void AddExecutor_Should_Add_User()
    {
        var project = ProjectTestData.CreateProject();

        project.AddExecutor(Guid.CreateVersion7());

        Assert.Single(project.Executors);
    }

    [Fact]
    public void AddExecutor_Should_Be_Idempotent()
    {
        var project = ProjectTestData.CreateProject();

        var id = Guid.CreateVersion7();
    }
}

```

```

        project.AddExecutor(id);
        project.AddExecutor(id);

        Assert.Single(project.Executors);
    }

    [Fact]
    public void AddExecutor_Should_Throw_When_Is_Manager()
    {
        var project = ProjectTestData.CreateProject();

        var id = Guid.CreateVersion7();

        project.AssignManager(id);

        Assert.Throws<DomainRuleException>(() => project.AddExecutor(id));
    }

    [Fact]
    public void RemoveExecutor_Should_Clear()
    {
        var project = ProjectTestData.CreateProject();

        var id = Guid.CreateVersion7();

        project.AddExecutor(id);
        project.RemoveExecutor(id);

        Assert.Empty(project.Executors);
    }

    [Fact]
    public void RemoveExecutor_Should_Throw_When_Has_Active_Tasks()
    {
        var project = ProjectTestData.CreateProject();

        var id = Guid.CreateVersion7();

        project.AddExecutor(id);

        project.CreateWorkTask("Task", null, DateTime.UtcNow.AddDays(1));
        var task = project.WorkTasks.First();

        project.AssignWorkTaskExecutor(task.Id, id);

        Assert.Throws<DomainRuleException>(() =>
            project.RemoveExecutor(id)
        );
    }

    [Fact]
    public void AddExecutor_Should_Throw_When_Project_Completed()
    {
        var project = ProjectTestData.CreateCompletedProject();

        Assert.Throws<DomainRuleException>(() =>
            project.AddExecutor(Guid.CreateVersion7())
        );
    }

    [Fact]

```

```

public void RemoveExecutor_Should_Unassign_From_Completed_Tasks()
{
    var project = ProjectTestData.CreateProject();

    var id = Guid.CreateVersion7();
    project.AddExecutor(id);

    project.CreateWorkTask("Task", null, DateTime.UtcNow.AddDays(1));
    var task = project.WorkTasks.First();

    project.AssignWorkTaskExecutor(task.Id, id);
    project.CompleteWorkTask(task.Id);

    project.RemoveExecutor(id);

    Assert.Null(task.ExecutorId);
    Assert.Empty(project.Executors);
}
}

```